Everything but the Software:

Feature-Driven Decay in an Agentic Coding Loop. An Exploratory Case Study and an Experiment Protocol

Juan Manuel Servera Bondroit*

Senior Partner Solution Architect, Microsoft EMEA, Zurich

Isabel Cabezas Martin

Software Engineer, Frontier Company at Microsoft

Draft of 8 November 2026

Abstract

Each feature changes the starting point for the next. In agentic software development, that starting point includes code, tests and the instructions guiding later work. This paper examines how faults and corrective rules accumulate in that inherited state. Maintenance literature and agent benchmarks distinguish conversational errors from repository-carried failures. A practitioner-led demonstrator supplies a motivating case: agent-assisted research, business and product requirements, architecture decisions, Design Thinking critique and disposable prototypes informed epics and user stories. Despite preparation intended to improve delivery speed and quality, extended autonomous work left the replacement unusable for the participant and prompted renewed feedback and refactoring. Five candidate mechanisms organise this transition: specification inflation, proxy capture, learning evaporation, process-gate inflation and feedback starvation. Migration and deployment dependencies remain central to the observed failure. A three-model, five-arm exploratory experiment is now running. It compares feature-only delivery, consolidation, read-only review, matched-resource delivery and visible regression repair, measuring final coverage, retained-contract loss, an unforeseen change request and resource cost. The selected 21 trajectories per model per arm give 315 planned trajectories, from the historical 12 (default), 21 or 33 options. The owner confirms an OSF-registered base dated 7 November 2026; current execution includes disclosed unregistered amendments. A coauthor conducts execution and data collection separately from study design and manuscript preparation to minimise potential experimenter bias. The study asks whether consolidation improves the system inherited by later feature work.

Keywords: agentic software engineering; coding agents; spec-driven development; software evolution; technical debt; refactoring; feedback loops; human–AI interaction; exploratory case study; experiment protocol.

1 Introduction

On 1 August 2012, Knight Capital's automated router sent more than four million orders while attempting to fill 212 customer orders. The firm lost more than USD 460 million. The SEC traced the incident to a defective function left in the router after a 2005 change, an incorrect deployment for a new trading programme, and inadequate controls. The new deployment activated old code that was no longer meant to run [87]. The failure crossed a feature boundary

*The views expressed are the authors' own and do not necessarily reflect those of Microsoft.

and a seven-year interval: the relevant starting point was the accumulated system, not just the latest patch.

Less spectacular versions of the same maintenance problem are familiar. Cunningham's 1992 account of WyCash described a financial product continually adapted to market demands. Its mature code was repeatedly revised and rewritten; the technical-debt metaphor explained why shipping a first-time implementation required later rewriting as understanding improved [26]. Some debt is visible as duplication or awkward dependencies. Some remains hidden in assumptions about inputs, deployment and earlier behaviour until another change depends on it. Counting completed changes reveals neither.

Agentic development makes this distinction newly pressing. PhotoCraft, an early-alpha reimplementation of Photoshop, reports 631 of 631 menu items live, but explicitly defines that count as dispatch rather than behaviour. Its own dated assessment estimates about 76% feature breadth and 45% readiness for real work [75]. A command can be reachable without the surrounding workflow being dependable. At the repository level, SWE-Maintain finds that agents solve fewer later tasks when starting from their own earlier changes than from the developers' changes, even though each task starts in a fresh session [69]. The next session can forget the conversation and still inherit its mistakes.

The engineering response has a long history: isolation and explicit contracts limit dependencies; tests and continuous integration expose regressions; small releases shorten feedback; refactoring preserves room for later change [7, 8, 31, 71]. Recent discussions by Beck, Martin and Fowler revisit these concerns for agent-assisted work [15, 32, 58]. Beck's distinction between producing code and learning through working software sharpens the research question [13, 15]: what does a feature leave behind for the next one?

The question arose in an architectural demonstrator developed by one practitioner with multiple agents. Research documents, business and product requirements and architecture decisions were built through sustained conversation, challenge and owner decisions. Design Thinking methods, including what-if scenarios, persona analysis and journey mapping, helped question the proposed solution; disposable pilots and prototypes exposed workflow problems before a usable MVP emerged. A backlog agent then converted that accumulated context into epics and user stories for a delivery coordinator. The intended advantage was faster implementation of better-understood features, supported by explicit acceptance criteria and review.

Extended autonomous sessions nevertheless left the replacement unusable for the participant. Renewed product-use feedback redirected implementation and refactoring. This sequence motivates investigating how preparation and local completion checks translate into the behaviour inherited by later work. The repository, pull requests, append-only log and versioned coordinator instructions retain the evolving delivery record [80]; the source package pins the evidence used here. The repository may be temporarily unavailable during the experiment embargo, after which the approved history and context are intended for release. The author's earlier experience with Aithena, a book-search project with PDF ingestion and a browser interface discussed separately in the supplement, and the PhotoCraft inspection selected after the motivating-case analysis have separate provenance in the ancillary anc/supplementary-context.md.

The working hypothesis is that feature-driven delivery can carry unresolved errors and avoidable change burden forward unless it includes cumulative verification and consolidation. Two kinds of state matter: the software being changed, and the instructions used to change it. Adding a corrective rule after every failure can leave prompts, skills and handover notes with their own maintenance burden. A fresh agent inherits both kinds of state. Section 2 distinguishes these pathways; the case then examines how they interact with completion proxies, review capacity and product-use feedback.

Here *feature-driven decay* describes loss of retained behaviour or increasing change burden across feature steps. Whether feature-driven decay occurs remains a question for the ongoing exploratory study.

Three questions guide the investigation:

- **RQ1.** Through which retained state can errors and change burden carry from one feature to the next, and what does existing evidence show about those pathways?
- **RQ2.** How does an exploratory delivery record illuminate that problem, and which candidate mechanisms and rival explanations does the gap between completion signals and usable replacement behaviour suggest?
- **RQ3.** Does scheduled agent-led consolidation after each feature improve retained behaviour and future changeability, and at what cost?

The report makes three linked contributions:

1. A focused synthesis distinguishing conversational self-conditioning, repository-carried faults and instruction maintenance, with a replot of published paired-history data (Section 2).
2. An exploratory account of the transition from agent-assisted discovery and challenged requirements to coordinated feature delivery and corrective product feedback. Five candidate mechanisms and within-case contrasts connect the interruption of usability to competing explanations (Sections 3–5).
3. A consolidation harness and an evolving five-arm exploratory protocol to estimate retained behaviour, future changeability and resource cost. Its earlier planning drew on the registered-report example of Lulla et al. [55] and HVE Core's Minimum Viable Experiment method (Sections 6 and 7).

The case concerns a migration and a configuration designed by Servera Bondroit, including delegated delivery and merge decisions. The case suggests hypotheses whose causal relationships remain to be tested. The primary comparison estimates a multi-component consolidation package against matched time and nominal-credit ceilings. The extra-allowance B–A comparison is secondary. Equal ceilings need not produce equal spending, and the design does not isolate refactoring or validate the five case mechanisms individually. An owner-confirmed registered base and disclosed unregistered amendments frame ongoing collection (Section 7.1). The case's validity and the pilot's identification limits are consolidated in Sections 5.2 and 7.6.

2 Background

2.1 Accumulation and inherited state

Lehman's laws and Parnas's software-aging account motivate maintenance alongside growth [51, 52, 71]. The 1997 formulation states the declining-quality law; it supplies no data for or against it. Eick et al. report mixed evidence from fifteen-plus years of change history [29], and a systematic review finds uneven support across systems [40]. Feature count and churn describe activity. A regression loses still-required, previously working behaviour; an unimplemented capability is an acquisition failure. A failure that continues across changes is persistence. Establishing that it caused a distinct later defect requires a trace of that dependency.

Cunningham describes shipping first-time code and rewriting it as understanding improves [26]. Fowler's design stamina is explicitly a hypothesis [31]. Reported debt costs concern different settings: a vendor-associated 39-codebase study reports 124% longer issue resolution in low-quality files [86]; 43 developers self-report about 23% of working time lost to debt [16]. These are not universal productivity coefficients.

What survives the next session? Code, tests and deployment assumptions survive through the repository; instructions and handovers survive through procedural context. Either channel can preserve a sound decision or a mistake. Here "software entropy" provides an engineering

analogy for the maintenance problem. The proposed measurements concern retained behaviour, adaptation and resource use.

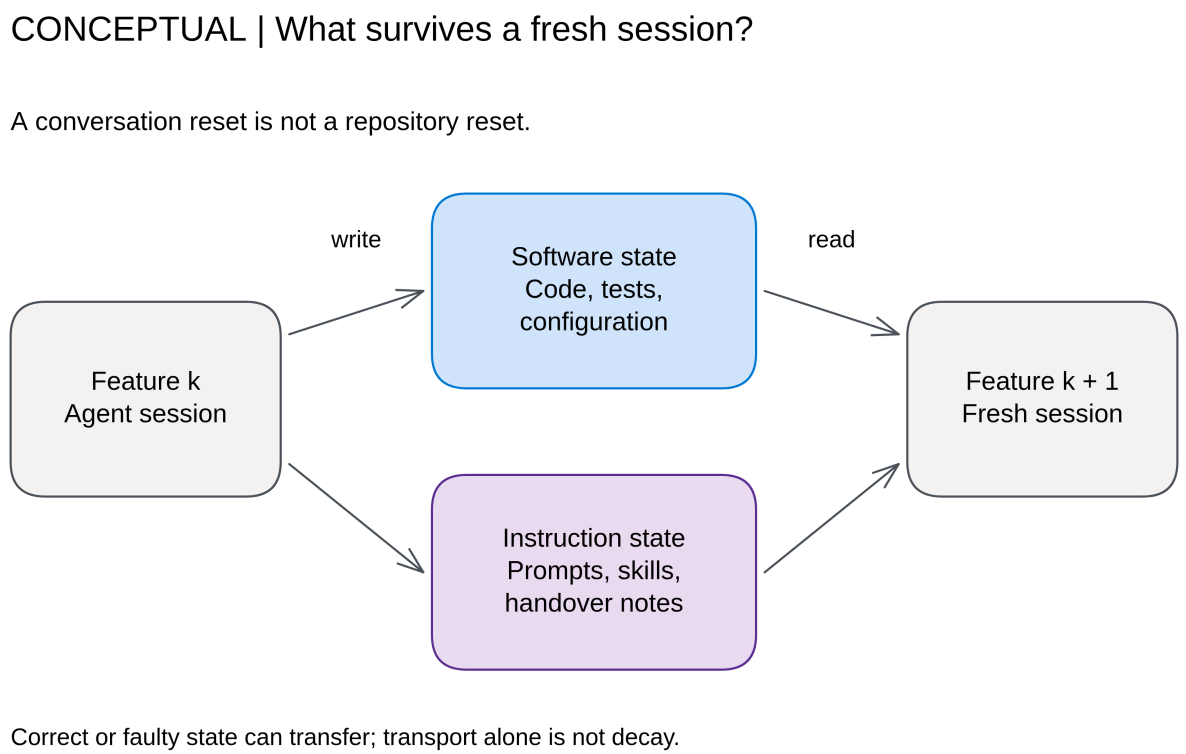


Figure 1: Conceptual read/write paths between sessions. A fresh conversation inherits software and instructions. Arrows identify state-carrying paths whose effects require measurement.

2.2 Evidence within and between features

Conversational benchmarks identify dependence on erroneous histories, not merely context length. Zhang et al. study false justifications after wrong answers [93]; Sinha et al. vary erroneous prior state updates in a controlled execution task [83, Secs. 2.2–3, Figs. 5–6]. Laban et al.'s 39% relative multi-turn performance drop concerns simulated underspecified conversations, not repository defects; locators refer to their arXiv v1 [49, Secs. 3.3, 6.1, App. F].

Pan et al.'s SWE-Maintain more directly tests repository inheritance: 100 three-event episodes from 50 Python repositories, with fresh sessions and matched later tasks under developer and agent histories. Across 1,996 paired later events, agent history reduces current-task resolution by 5.6 points; requiring earlier tests also to pass gives a 14.0-point gap. Unweighted mean current-task gaps grow from 4.0 to 7.2 points between the second and third events. In the selected 619-event clean-history subset, the current-task difference is -1.1 points, interval $[-3.8, +1.6]$, and cumulative event resolution differs by $+2.7$, interval $[-0.5, +6.1]$ [69, Secs. 4.1–4.2, Tables 2–3]. This favours inherited behavioural failures over complexity alone as the main observed channel, without establishing a general decay law or a randomised repair effect.

Their `python_unidiff` example reuses line-splitting logic that loses terminators in a later constructor; Figure 1's caption introduces it and Section 5.1/Figure 5 supplies the detail [69]. SlopCodeBench and SWE-CI extend iterative evaluation [23, 68]. Borg et al.'s maintainability study finds no significant downstream difference for late-2024 assistants; only phase two was randomised [19]. Its findings provide a useful contrast across tool generations and collaboration settings.

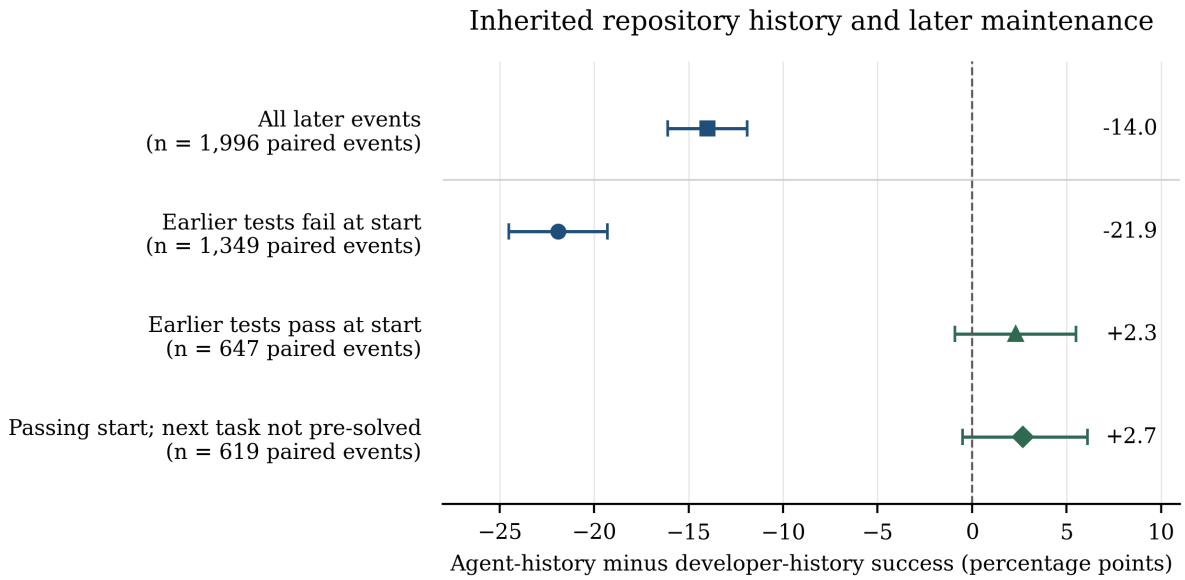


Figure 2: Published event-resolution differences and repository-clustered 95% intervals from Pan et al., Table 3. Current and prior non-superseded tests must pass. The overlapping selected cohorts apply different restrictions to the same benchmark. Points and whiskers reproduce the published estimates and intervals.

2.3 Instruction maintenance and evaluation

Instruction files themselves require maintenance [67, 76]. Mohsenimofidi et al.'s 169 commits have 242 overlapping assignments across nine categories; 78 instruction additions, 59 modifications and 23 removals are three categories, not a partition [63, Sec. 4.3, Table 2]. Chatlatanagulchai et al.'s sampled maintenance labels likewise include addition and pruning [21]. Instruction debt here is the author's term for obsolete, duplicated, conflicting or poorly routed directives.

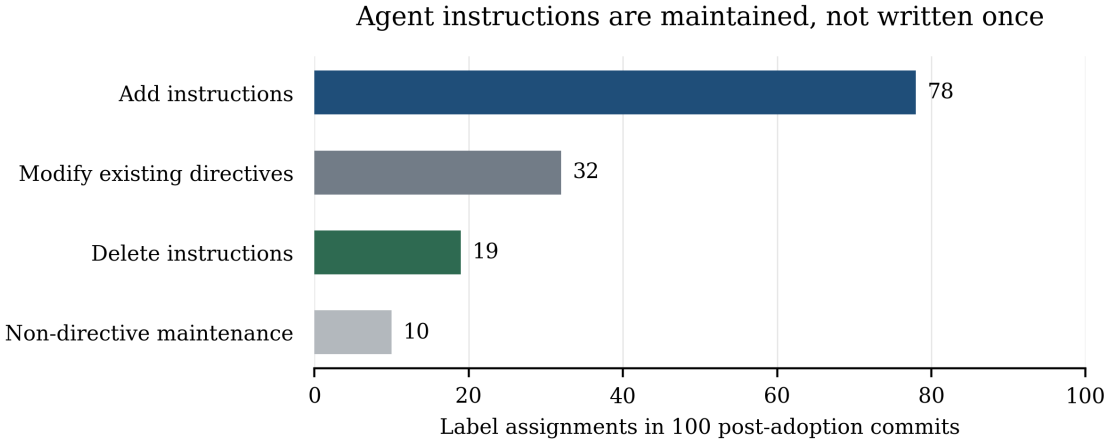


Figure 3: Labels in 100 sampled post-adoption commits, from Chatlatanagulchai et al., Sections 4.2.2–4.2.3. Labels overlap and count edit types, not net token growth or effects on agent success.

Gloaguen et al. report 20%/23% higher inference cost with LLM-written context and slightly lower success, by 0.5/2 points, without significant differences from no-file baselines. Developer-written files outperform LLM-written ones on CTXbench ($p = 0.038$) [36, Sec. 4.2, Tables 2–3]. SkillOpt and Jiang et al. provide counterevidence to treating retained instructions as inherently harmful: evaluated procedural guidance can help, while retrieval and downstream success remain

distinct [44, 62].

Tests and review. RefactorBench studies refactoring, not between-feature consolidation [35]; developer-agent collaboration studies are observational [48]. SWE-Sharp-Bench supplies reproducible repository evaluation [59], while CRBugBench exposes review limits [90]. Building to the Test finds a dead or absent required library in seven of twelve oracle-in-the-loop runs [56]. ImpossibleBench tests exploitation more broadly [94]. Test integrity and adequate outcome coverage are different requirements. AgentRx motivates traceable failure diagnosis [60].

Bounded correction. Pavlenko et al.'s factory optimises human-architected systems against measurable objectives; productivity estimates cover application work, excluding infrastructure and onboarding [72]. AutoSaddler treats harness updates as offline learning [70]. Neither isolates the present intervention and later change probe.

Reflection and self-improvement. Ng's reflection design-pattern letter explicitly recommends Self-Refine, Reflexion and CRITIC [66]; he is not a coauthor of those papers. Self-Refine iterates feedback and revision with the same model without training [57, Sec. 2]. Reflexion retains linguistic feedback in episodic memory for subsequent trials rather than updating model weights [81, Sec. 3]. CRITIC uses external tools to check and revise an output [38, Sec. 3]. These are precedents for reflective correction, not evidence that every retained lesson or revision improves later feature work. Their feedback and repeated-trial budgets differ from the present protocol, whose hidden evaluator outputs never reach agents.

Ng coauthored MedAgentBench, which evaluates task completion in an interactive medical-record environment rather than chatbot answers alone [43, arXiv v2, Secs. 2.3–2.4]. Its evaluation of observable action outcomes informs the measurement focus of this protocol.

The two owner-selected videos illustrate these boundaries rather than provide experimental findings. In the Ng/Moroney talk, Ng describes human-feedback iteration; Moroney later warns about code structure and describes task-level reflection [74]. In the separate memory presentation, a speaker identifying herself as Anthropic staff describes persistent memory and asynchronous transcript-driven curation, with human review of proposed updates [5]. Both sources are third-party uploads checked through automatic captions, not audio-verified research reports. Neither supplies a controlled cross-feature consolidation result.

Engineering rationale. Beck's alternation of features and tidying aims to restore future options [10–12, 15]. Extending that metaphor to acceptance criteria is this author's interpretation. Understanding/abstraction and learning through use have separate sources [14, 15]. Tests, short releases, explicit design responsibilities and usable increments motivate the checklist [6–8, 20, 24, 33, 34, 42, 58, 65, 78]. Monitoring burden and completion proxies qualify the rationale [2, 25, 37, 50, 82, 84, 85]. Spec-driven tools structure handoffs; the intended outcomes still need execution checks [1, 18, 27, 88].

HVE Core. Microsoft's Hypervelocity Engineering Core supplies file-based Research–Plan–Implement–Review handovers; the author's coordinator added autonomous dispatch. The reported case snapshot is `plugins-v3.3.106`, not proof of its installation date [61]. Additional source qualifications and reproducibility detail are in the supplement.

3 Case and method

3.1 Setting

The motivating project was an architectural training exercise developed by one practitioner acting as architect, developer and product owner. Its synthetic, multi-role service workflow combined documentary evidence, approvals, identity, data and external-service dependencies. The exercise required research, requirements, architecture, a delivery plan and a working demonstrator. These interdependencies made preservation of a complete user journey consequential even in a non-production system. No operational service or real resident data is evaluated here.

Agents performed most research synthesis, drafting and coding, with the practitioner steering scope and decisions. Several weeks of discovery and requirements work preceded roughly a week of interactive MVP development and a ten-day autonomous delivery window. The project continued afterward; its current state and detailed chronology belong to the versioned repository record [80]. An automated repository scorer supplied an additional completion signal. Internal assessment wording is not reproduced.

3.2 Workflow and tools

GitHub Copilot and HVE Core supported a sequence of connected activities:

- Research agents assembled a knowledge base of source snapshots, maintained syntheses, an index, owner-curated memory and an append-only log.
- The Design Thinking coach and stakeholder-role agents challenged assumptions through what-if scenarios, persona analysis and journey mapping. Their critiques separated bounded AI tasks from deterministic controls and accountable human decisions. Role-agent answers were hypotheses for the exercise; direct stakeholder research remained a separate requirement.
- BRD and PRD builders, quality review and the ADR assistant turned successive discussions and owner decisions into revised requirements and architecture records. These documents changed as understanding developed.
- Disposable pilots and prototypes tested candidate workflows. Their critiques and participant feedback informed a deployed MVP that the author used intermittently and refined through short feedback loops.
- A backlog agent converted the accumulated context into epics, features and user stories, with acceptance criteria, dependencies and indications of existing MVP coverage. Some stories also exposed scope needing subsequent requirements reconciliation.
- A custom coordinator then dispatched implementation and review work against that backlog (Section 3.3).

The intended benefit was accelerated planning and better-grounded feature delivery. Neither a human-only comparator nor a measured speed or quality gain was available. The retained documents and conversations matter because they shaped what workers were asked to build and reviewers were asked to accept.

3.3 The coordinator

The coordinator ran through Copilot CLI on a VM. It selected dependency-ready issues and launched parallel worker sessions in isolated git worktrees. Each worker received an issue and HVE Core's research, plan, implement and review (RPI) workflow. The coordinator checked the resulting pull request, updated the project board and, under owner delegation, merged changes. Interactive and CLI model configurations changed during the project; recorded selectors

and recalled interface labels are distinguished in the source package. The delivery record is consequently an evolving workflow.

Early worker sessions implemented directly, prompting the author to make the full RPI sequence and committed tracking artefacts mandatory. Further instructions required evidence-backed acceptance ticks, scheduled monitoring and increasingly comprehensive local gates. Human decisions, approvals, credentials and unresolved review findings could still block work. At the same transition, an accepted architecture decision prioritised migration to a new source layout and froze feature work in the MVP. Coordination, migration and procedural requirements therefore changed together.

3.4 Data and analysis

This exploratory single-case study follows the transition from interactive discovery and MVP use to backlog-mediated autonomous delivery. The author selected the case after encountering the unusable replacement and acts as participant-observer. Case-study guidance and the ACM SIGSOFT Empirical Standards inform the account [77, 79].

The original analysis pins repository commit `29c6b51`; a separately pinned follow-up uses `9b74ac8d`. Sources include the knowledge-base log and index, requirements, architecture decisions, coordinator definition, issues, pull requests and retained RPI records. The author's review adds process clarification and an export of project issue states, retained as anc/project-insights-2026-11-01.csv. Source dates, historical counts and extraction rules remain in the ancillary records. The main account uses them to reconstruct the sequence and handoffs.

Analysis combined descriptive measures, reference tracing of recorded lessons and AI-assisted interpretation of selected episodes. It did not include systematic entry-by-entry coding. Early log entries were reconstructed from history; later entries are contemporaneous project accounts. Specification review was selective. Participant recollection, committed source and execution evidence remain distinct, including in the author's fuller account of renewed feedback. The original source package retains missing score-to-release and deployed-journey links.

External repositories and literature were selected for relevance to inherited faults, instruction maintenance and outcome verification. They supply contextual comparisons and a focused narrative synthesis. The published-evidence figures reproduce reported values with source locators; the external applications and original studies were not rerun. The ancillary observation record preserves its purposive issue sample, exclusions and retrieval boundaries. These sources are analysed separately from the motivating project.

3.5 Constructs and evidential boundaries

Here an *agent* is a configured model–tool–instruction loop that selects and executes actions, not the model alone or an independently accountable person. A *skill* is a reusable procedural resource describing when and how to act and verify the outcome, distinct from an execution history or a model-weight update. This procedural distinction follows Jiang et al. [44, Sec. 1]; the pilot's handover note is retained context, not a separately manipulated skill treatment.

Working behaviour means completing an intended task through the running system, including the identity, data and service dependencies needed for that task. A healthy endpoint or a passing repository test alone does not establish it. *Decay* denotes loss of previously working behaviour or increasing difficulty of understanding and changing a system. The case records lost usability during replacement; any decline in intrinsic code quality remains unmeasured. Deployment drift could produce the same observation. *Adaptability* is operationalised prospectively as success and cost on the fixed-budget, unforeseen change request, not as a universal measure of maintainability. *Consolidation* means scheduled structural work intended to preserve observable behaviour, with tests checking that intention; passing finite tests cannot prove semantic equivalence.

Five measurement categories remain separate. PRs, lines and words describe *production volume*; checks, reviews and receipts describe *validation activity*; observed user tasks describe *system behaviour*; code metrics and the change request probe *internal quality and changeability*; and corrective help requested or supplied describes *intervention*. The original observations cover the first three. A later post-hoc comparison adds structural proxies, not a controlled changeability task or systematically coded intervention dataset. Neither volume nor validation activity is used as a direct estimate of decay.

3.6 Chain of evidence and reflexivity

Table 1 separates observations, interpretations and proposed checks. The relevant evidence concerns how discovery was carried into implementation, how completion was judged and how product feedback returned. Repository sources refer to the original case pin unless a later record or participant account is identified. Detailed locators and unresolved execution links remain in anc/case-evidence-matrix.json.

Table 1: Evidence relevant to the delivery transition and checks that would distinguish the candidate explanations.

ID	Observation and provenance	Interpretation	Discriminating evidence
O1	The log and participant account describe a used MVP followed by a replacement lacking identity, staff and model prerequisites.	Delivery failed to preserve a complete working journey.	Execute the same journey against versioned deployments and configurations.
O2	Pull requests and automated assessments recorded completion while participant sign-in remained blocked.	The completion signals covered a different or insufficiently linked outcome.	Link the assessed capability and release to the failed deployed task.
O3	Requirements and process records expanded through discovery and delivery; even a small source correction carried a substantial evidence record.	Documented context may outgrow what a worker or reviewer can use effectively.	Compare actual context access and review effort with validated behaviour.
O4	Simplicity and deployment lessons were recorded, but later replacement work omitted previously working prerequisites.	Lesson availability and application may diverge.	Trace the relevant session inputs and whether the lesson was actionable or deliberately deferred.
O5	Coordinator rules formalised gates and delegated merging; the author continued resolving blockers and later requested stabilization.	Approval activity and product-use feedback served different functions.	Compare outcome checks, steering decisions and subsequent implementation changes.

Servera Bondroit selected the episodes, designed the coordinator and had an interest in successful delivery. Those roles favour a coherent retrospective explanation. Experience-based beliefs can differ from project evidence [28]; the same caution applies to AI-written records and AI-assisted synthesis. Independent checking of O1–O5 against retained source and execution records remains outstanding. Jiang et al. make mechanism analysis inspectable through matched trajectories, evidence-linked labels and a separate human check of taxonomy construction, while distinguishing that check from exhaustive human coding [44, Sec. 4, App. A.2]. Here the five mechanisms organise selected episodes. Their prevalence and causal effects require a different design. The evidence index retains null release or execution identifiers where a link still needs checking.

4 Exploratory testbed: observations

The relevant sequence is the movement from exploratory dialogue to a formal backlog, autonomous issue delivery and renewed product feedback. Table 2 summarises those phases. The detailed project chronology and changing totals remain in the repository and source package.

Table 2: Phases of the motivating case and the state carried forward.

Phase	Activity	State inherited by later work
Discovery, several weeks	Research, role-agent and Design Thinking challenge, what-if scenarios, persona and journey analysis; iterative BRD, PRD and ADR work.	A rich but evolving requirements and decision context.
Interactive probes, about a week	Disposable prototypes, participant critique and short implementation/feedback cycles produce a used MVP.	Working journeys, simplicity lessons and deployment prerequisites.
Backlog handoff	A backlog agent derives epics and user stories, criteria, dependencies and existing-coverage labels.	A formal work hierarchy that selects implementation and review tasks.
Autonomous delivery, about ten days	Parallel RPI workers deliver issues under a coordinator while migration and gate requirements evolve.	Merged changes and validation records; the replacement lacks a complete usable journey.
Renewed steering and follow-up	The participant prioritises usable increments, functional checks, refinement and refactoring; stabilization is formalised.	Product-use feedback again shapes what subsequent work must establish.

4.1 From discovery to epics and user stories

Preparation involved sustained steering rather than a single specification prompt. Role-agent challenges questioned the requested end-to-end agents; what-if scenarios tested alternative inputs and responsibilities. Persona and journey work exposed handoffs, missing information and approval boundaries. The resulting BRDs, PRDs and ADRs were critiqued and revised through further conversation. These activities changed the proposed solution before and during implementation.

Disposable prototypes then made some assumptions tangible. Participant critique and an agent-assisted simplicity review produced requirements for clear ownership, a visible next action, understandable blockers and manual paths that remained usable without AI. Short feedback cycles reshaped an MVP around those lessons. The author reports using it intermittently, including sign-in, synthetic data and model-backed assistant paths.

The backlog manager drew on this accumulated context to generate the work hierarchy, acceptance criteria and dependencies. Existing-coverage labels distinguished proposed work from functionality already present in the MVP. The supplied project export records a batch expansion around this handoff, followed by movement into completed states. It provides a trace of work organisation; accepted user journeys require their own evidence. Requirements reconciliation continued afterward, including for scope first surfaced in backlog stories. Thus preparation and delivery overlapped, and the issue hierarchy remained an evolving interpretation of the design.

4.2 Local completion and the usable workflow

Autonomous delivery changed the feedback path. The coordinator selected issues and workers satisfied local implementation and review obligations. At the same time, a source-layout migration replaced the used MVP. The replacement passed bounded engineering checks, including health, restart and unauthenticated-request rejection, but lacked identity, staff and model prerequisites needed for the participant's complete workflow. Restoring a public entry point left sign-in dependencies unresolved.

The author continued supplying approvals and resolving technical blockers. What diminished was the ability to exercise realistic work in the replacement and use that experience to steer the next change. Missing prerequisites became further issues, each with research, planning, review and correction work. Scope additions and validation costs also changed delivery conditions. These observations locate a concrete interruption of usable behaviour within a broader process of accumulating obligations. The automated score and failed journey still need release-and-scope linkage before their disagreement can be assessed as a matched comparison.

4.3 Bounded within-case contrasts

Table 3 compares episodes and negative cases already in the record. Migration, models, workload and gate rules changed alongside coordination, so these contrasts motivate explanations for later testing.

Table 3: Contrasts relevant to context, completion and feedback.

Contrast	Recorded difference	Analytical consequence
Interactive MVP versus autonomous replacement	Product use supported short feedback cycles; the replacement could not support the same task.	Feedback is a candidate explanation alongside the changed migration and environment.
Requirements before versus during autonomy	Expansion and review obligations already existed during discovery.	The burden has a longer history than the coordinator, and some detail preserves important controls.
Engineering checks versus participant task	Bounded checks passed while real-user sign-in remained blocked.	Acceptance needs coverage of the whole journey and its deployment dependencies.
Exploratory challenge versus backlog execution	Role agents questioned the initial proposal; delivery workers implemented specified stories.	Agent behaviour depended on task framing, context and opportunities for challenge.

4.4 Renewed feedback and follow-up

The participant requested a usable increment, prioritised feedback-first work and asked the coordinator to pause, exercise the accumulated functionality and refactor. The participant's account places this renewed feedback at the turning point in making the replacement useful. Subsequent source history records implementation refinement and recurring stabilization checkpoints. The retained checkpoint review qualifies its local engineering evidence and leaves live acceptance separate.

This follow-up matters because the project was steered through interaction: the backlog and the worker instructions did not settle the product outcome once and for all. A dated, release-linked journey is still needed to measure time to restored usability. The original observation window and the later repository snapshots remain distinct. The repository supplies the evolving history; the ancillary records preserve the exact observations used here, including changing issue totals, document volume, lexical counts and static source metrics. Those records support reconstruction while the prospective experiment supplies a separate test of retained behaviour and changeability.

5 Five candidate mechanisms

Five candidate mechanisms organise the gap between extensive preparation, local completion signals and an interrupted usable workflow. Table 4 summarises the indicators; the supplementary operational audits set out definitions, counter-evidence, rivals and discriminating checks. Together they connect the inherited-state problem to the way this delivery loop selected work, carried lessons and checked outcomes.

Table 4: Five candidate mechanisms, case indicators, relevant prior work and proposed early-warning signals. Indicators are not causal tests.

Candidate	Case indicators	Related prior work	Early-warning signal
Specification inflation	Expanding requirements and acceptance obligations during both discovery and implementation	Royce; Gall; Brooks; Böckeler; Beck	Specification growth versus human-validated behaviour
Proxy capture	Artefacts became a merge condition; completion assessments coexisted with blocked sign-in, with release/scope linkage still pending	Goodhart; Strathern; Beck's genie; ImpossibleBench; METR	Artefacts per behaviour change; scores that disagree with a live demo
Learning evaporation	Recorded simplicity and deployment lessons were followed by missing prerequisites; actual session access remains unknown	Naur; context studies; multi-agent failure taxonomy	Lessons that exist only as prose; problems solved twice
Process-gate inflation	Broad per-push gates, constrained validation resources and an exception for a small correction	Bainbridge; ironies of generative AI; Murphy; Goldratt	Gate time per change; review minutes per merge
Feedback starvation	A used MVP was frozen; the participant could not exercise the replacement and requested usable increments	XP; incremental delivery; tracer bullets; Agile Manifesto; DORA	Time to first human use; age of last usable increment

They occupy different points in the delivery process (Table 5). Specification growth increases the obligations and interactions that later work must preserve. Learning evaporation concerns the path by which a relevant lesson reaches the next session. Proxy capture concerns what selects and approves work; process-gate inflation and feedback starvation concern the capacity to detect and correct cumulative problems. None needs to be called error propagation to explain how an inherited fault can survive.

Before the participant's checkpoint request, the loop had no scheduled cumulative consolidation step. Its large restructuring, the source-layout migration, was planned as backlog items; local refactoring could still occur within individual issues. The experiment separates retained-contract loss, structural fidelity diagnostics and later adaptation. Formal fault-lineage annotation would address the stronger question of whether an earlier fault generates distinct later failures; it is deferred rather than added to the pilot's workload.

The five categories in Table 4 are post-hoc interpretations. The supplement states their operational boundaries and the checks needed to distinguish their effects.

5.1 A proposed reinforcing relationship

The mechanisms suggest a possible reinforcing loop (Figure 4). In this case a large specification accompanied a large backlog and parallel sessions. Artefact-based gates generated more documents; some recorded lessons were absent from later briefs; regressions were not exposed through routine product-use feedback. The author responded to gaps with more rules, criteria and gates. That sequence is consistent with reinforcement, but does not establish each causal link. A running increment and a human checkpoint are proposed conditions of every cycle, to

Table 5: Accumulation, transmission and correction in the working model.

Process	Question	In this case
Accumulation	What obligations and change burden does each feature leave behind? [51, 52]	No scheduled cumulative consolidation before the checkpoint request; restructuring planned as backlog items
Transmission	Which implementations, rules and lessons shape the next change? [69, 83]	Growing specifications and read path; previously working identity setup not preserved
Selection and correction	Do review and tests expose the cumulative consequences before more work depends on them? [25, 92]	Proxy capture; process-gate inflation; feedback starvation

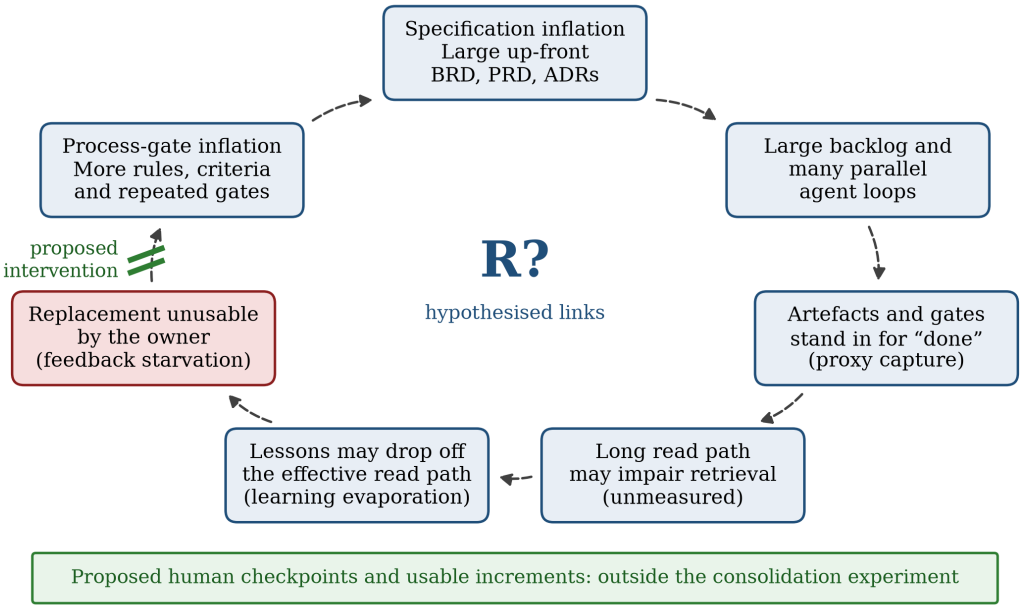


Figure 4: Proposed reinforcing relationships drawn from the case interpretation. Their causal effects remain unmeasured. Context overload is unmeasured. The proposed human-checkpoint intervention differs from the consolidation package tested in Section 7.

test whether they interrupt it.

5.2 Case validity and rival explanations

The five categories are not independent causes identified by these data. The same missing identity and staff bindings can indicate deferred migration work, incomplete deployment acceptance or a lost lesson. Discriminating them requires the deployment comparison and actual session-input trace identified in Table 1, not another count of documents or citations. Similarly, a growing read path does not demonstrate context overflow, and an expensive gate is not necessarily a disproportionate gate. Per-change gate time, avoided defects and human effort were not measured.

The source-layout migration, owner-selected priorities and delegated merging changed alongside coordination. Scope additions, budget constraints, model/tool changes and deployment conditions also differ between periods. These are direct rivals to an autonomy-only account. The supplementary rival-explanation register retains the full register, separating recorded circum-

stances from plausible but untested alternatives. Bird et al.'s study of co-installed applications on Windows machines supplies a methodological analogy for checking environmental influences on reliability [17]. It neither tests AI-agent configuration nor diagnoses this case's sign-in failure.

Construct validity is limited by whitespace counts, lexical indicators and selectively reviewed records. Appropriate caution can raise boundary-phrase density; missing references do not prove missing knowledge. As coordinator designer, participant and analyst, the author selected the episodes and may favour a coherent retrospective explanation. AI synthesis is not an independent observer. Public records support recomputation but not independent corroboration of every statement. The single practitioner-designed exercise and purposively selected external examples do not establish prevalence or general causal effects. The proposed experiment has separate identification limits in Section 7.6.

6 Consolidation hypothesis and harness

The observed operational failure and the prospective hypothesis are distinct. Missing deployment identity and staff bindings are concrete obstacles to sign-in; structural code consolidation would not necessarily restore either. Recovery of that journey requires configuration and end-to-end verification. Consolidation is worth testing for a different reason: repeated feature work may make later changes harder, even when deployment dependencies are correct. The experiment probes that possibility through retained tests and CR1; it does not claim that consolidation would have prevented this episode.

The idea is already practised. OpenAI describes replacing weekly manual clean-ups with recurring background agents that open small refactoring pull requests, alongside a documentation-gardening agent [54]. Anthropic asks each session to leave a clean state and update a progress file [91]. Neither account reports a controlled comparison of feature-only delivery with consolidation between feature steps. Darwin's continuous optimisation is a further precedent for automated improvement, but its performance evaluations do not isolate that intervention or test an unforeseen change request [72] (Section 2.3). RefactorBench already evaluates refactoring agents [35]; the narrower hypothesis concerns retained behaviour and later adaptation during repeated feature additions.

AutoSaddler's preprint abstract already treats harness improvement as offline learning from execution traces, combining failure diagnosis, structured patches and validation-based update selection [70]. Agent-led reflective correction is therefore not the novelty claimed here. The proposed intervention instead consolidates the retained repository between feature steps and measures retained contracts and a withheld change request.

The proposed consolidation harness tests a bounded part of reflective correction. The distinction is what persists afterward. Self-Refine revises an output and Reflexion retains lessons for later trials [57, 81]; the present intervention instead consolidates the software and its handover between different feature requests. CRITIC's tool feedback is another precedent [38], but it does not authorize feeding the held-out evaluation into this experiment. Whether the retained changes help an unforeseen request remains the empirical question. The harness schedules each pause; the agent is not authorised to reject features or redirect the product. After every feature the loop stops, and an agent:

1. runs the tests and a metrics script that lists complexity hotspots, duplicated blocks and lint warnings;
2. reviews the last feature's diff and the hotspots against a short checklist (duplication, long or complex functions, unclear names, dead code, rules from one feature leaking into another, tests that check implementation rather than behaviour);
3. refactors in small, behaviour-preserving steps, each applied through `test && commit || revert` [9], so that every change either keeps all tests green or disappears;

4. may add tests that pin current behaviour, but may not change or delete the example tests that define the features;
5. rewrites a design note of at most 300 words that says where each rule lives and what debt remains, replacing the previous one rather than appending to it;
6. stops when a quality budget is met or its time budget runs out.

Only then may the next feature start. The supplement reproduces the historical v4 prompt; the current researcher kit defines the amended prompt. The active B/C templates and their source bindings are reproduced in Supplement Section A.10. The intervention combines refactoring, additional tests and a design-oriented note; it is not a pure refactoring treatment.

The hypothesis draws on Beck's alternation of features and futures [12, 15] and *Tidy First?* [10]. It is not free, and may not work. Agent refactorings can be incomplete or incorrect [35]; human studies also caution against equating a refactoring label with a quality gain. Kim et al. report developers' refactoring challenges and benefits [47]; Bavota et al. find that observed refactorings do not consistently target or remove code smells [4]. These findings motivate checking outcomes rather than assuming improvement. Test-commit-or-revert protects only behaviour represented by its tests, not every intended contract [56]. Section 7 specifies how the package will be evaluated.

7 Testing the harness: an evolving exploratory experiment

The experiment asks whether scheduled correction improves the repository inherited by later feature work. RQ3 is addressed by a five-arm exploratory study now in execution. Agent configuration protocols and empirical-study guidance informed its earlier planning [3, 55, 89].

7.1 Execution status and disclosed amendments

As of 8 November 2026, Isabel Cabezas Martin is conducting the experiment and collecting its data independently under the predefined exploratory protocol and disclosed amendments. Study conception, methodology, evaluation criteria and manuscript preparation were undertaken by Juan Manuel Servera Bondroit. This operational separation is intended to minimise potential experimenter bias; both authors are Microsoft employees, and neither institutional independence nor independent scientific review or analysis follows from it.

The owner confirms an OSF-registered base at https://osf.io/xcpt4, dated 7 November 2026. Its metadata was not independently verified (Data availability). Current execution includes disclosed unregistered amendments: short calibration, parallel execution and VM restarts. The run is an exploratory pilot/estimation study, rather than a single preregistered confirmatory evaluation. The public protocol's empty formal hypothesis list and false/null current-amendment registration fields do not negate the owner-confirmed registered base.

The active profile is `pilot-estimation.json`, `model-stratified-v2-five-arm`, suite `s1-retrofit`. GPT 6.1 Sol, Claude Sonnet 5.5 and Gemini 4 Argon use medium effort across arms A–E, F1–F10 and held-out CR1, with seed 2027. The owner-supplied runtime selection is $n = 21$ trajectories per model per arm despite the file default of 12: $3 \times 5 \times 21 = 315$ planned trajectories. Feature steps, sessions and completion counters do not supply an independent sample size or a completed-trajectory count.

Collection spans three implementation epochs, ending at commit e71037d961cb80fc5c6594 37796de7736820f6c9, without a freeze tag. Data availability identifies all three sources and the owner-reported step counts. Failures, unknowns and epoch labels remain part of the record; implementation changes and execution scheduling can be confounded with time, arm and model. The dataset cannot be treated as one homogeneous frozen implementation. Commit times locate source changes, not collection events.

The 1 November five-arm amendment, 2 November allocation revision and 5 November v6 source findings below remain dated planning and inspection records. Historical v4/v6/v7 freezes and diagnostics retain their own scope. Earlier power/resource scenarios, including full ten-feature calibration, are preserved unchanged; they do not establish power or expenditure for the active F10/$n = 21$ amendment.

7.2 Design and estimands

One independent replicate is a fresh repository trajectory through S1's ten ordered features and a final held-out change request, CR1. Every selected model, GPT 6.1 Sol, Sonnet 5.5 and Gemini 4 Argon, runs every arm. Randomised interleaving within model/replicate blocks uses seed 2027. Feature steps, tests and checkpoints are repeated observations within a trajectory, not independent replicates; agent sessions are not independent units either. The selected number n applies *per model per arm*, equally across all fifteen cells.

Table 6: Five-arm comparison retained from the earlier design. D is the matched-resource A+ control; E tests visible regression repair, not hidden-test feedback.

Arm	Intervention and information	Per-cycle ceilings
A	Feature delivery only	30 min; 300 nominal credits
B	Feature, then behaviour-preserving consolidation, additional tests and replacement DESIGN.md	30 + 15 min; 300 + 300 credits
C	Feature, then read-only review and replacement REVIEW.md	30 + 15 min; 300 + 300 credits
D (A+)	Longer feature delivery, without a scheduled consolidation pause	45 min; 600 credits
E	Feature, then repair of previously specified behaviour using visible specifications and examples	30 + 15 min; 300 + 300 credits

The 5 November source check identified all five arms and an immutable `-replicates` selection of 12, 21 or 33 (default 12) in the source-published v6 kit. D uses the longer feature session without a post-step; E implements visible-specification repair. Data availability lists the inspected execution and manifest commits. These source findings supersede the earlier A/B/C-only capability account. The v6 release records that historical implementation; its then-uncompleted registration and collection gates are distinct from the later registered base and unregistered execution path. Its source-declared runtime qualification was not reproduced by this review. The historical v4 freeze remains unchanged.

B–D is the primary practical comparison: consolidation versus giving the builder the same total time and nominal credits. Equal ceilings do not guarantee equal realised resources; D also has fewer fresh sessions. B–A measures the additional package; B–C also changes editing rights, tests and handover content. E–B contrasts visible behavioural repair with behaviour-preserving consolidation. These secondary package comparisons do not isolate refactoring or any of the five case mechanisms.

C cannot repair code during its review session, but REVIEW.md can direct repairs in later feature sessions. Follow-through needs review-item and diff locators with external adjudication; file overlap alone is not proof. B's added tests must agree with visible specifications, rather than legitimating a defect. Subsequent feature sessions must keep accepted tests green. E can correct an agent-authored expectation only with a specific contradictory specification and a retained explanation. Accepted Python test files are immutable in full, including imports, fixtures and module/class constants; additions use new files. E retains only its explicit specification-bound correction exception. Immutable example tests remain protected in every arm.

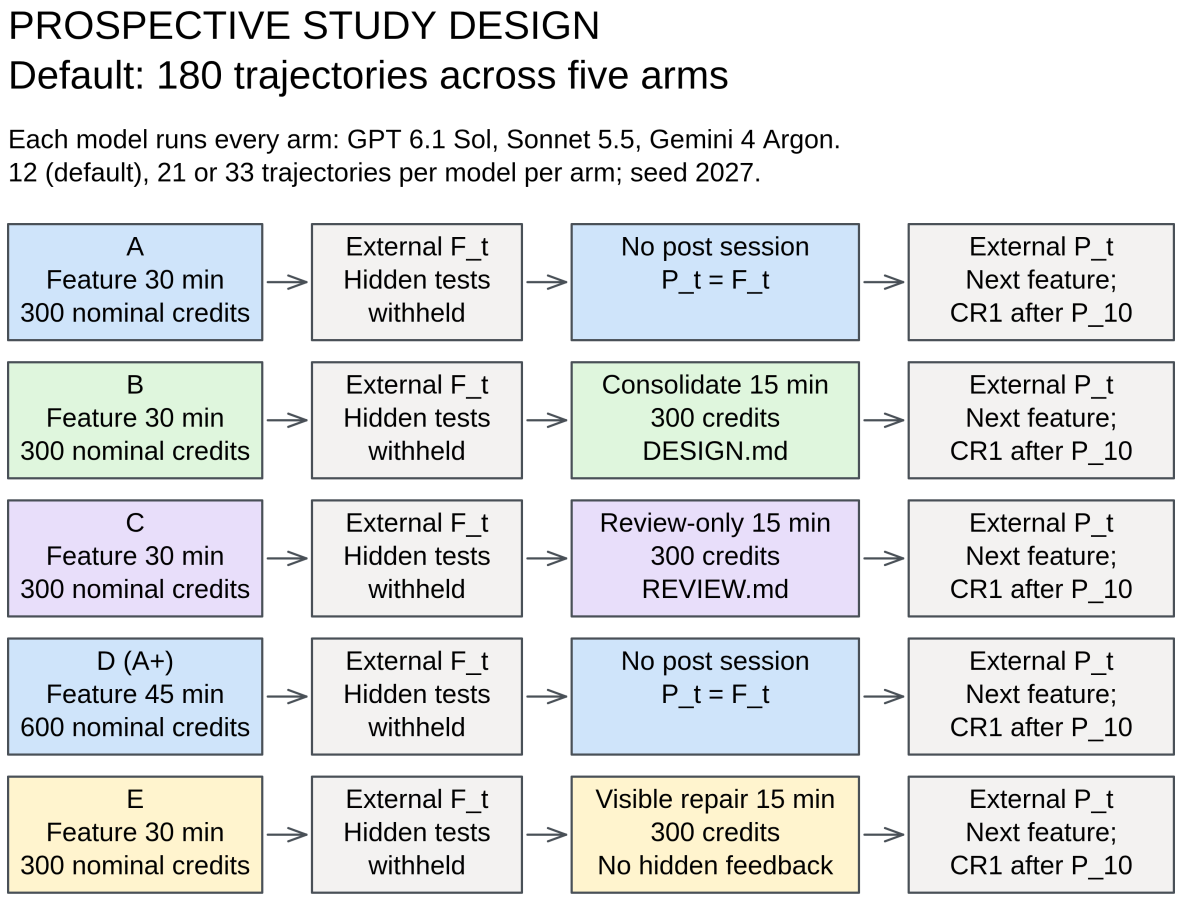


Figure 5: Historical v6 design diagram. Its default allocation, full calibration and pending-registration/launch labels describe that source record; current $n = 21$ execution and amendments are in Section 7.1. Every model runs all five arms. The external evaluator is separate from candidate tools. CR1 content remains withheld until its session; privacy and isolation reduce exposure without proving that a model has no prior knowledge.

7.3 Development and evaluation execution boundaries

The pinned runtime separates model transport from candidate-code execution (Figure 6). The trusted host runs the Python experiment controller and JavaScript Copilot SDK/CLI transport. Prompt preparation and the authenticated model connection remain on that host. The development model can request `isolated_exec`; a Python broker runs each requested command in a fresh isolated container, returning bounded stdout, stderr and exit status through the transport to the model. Validated filesystem/Git state is replayed into a fresh writable isolated workspace for the next call; this is not a persistent container or writable host checkout. Candidate processes and memory do not persist, while host/model conversation context is not reset by that boundary. Only validated committed regular content crosses feature/CR checkpoints; the host canonicalises history rather than retaining arbitrary actor commit metadata. Visible-test verification and hidden measurements use separate evaluator containers after development steps.

The supplementary post-run path begins with a stopped local study archive. Host Python selects recorded checkpoint HEADs and asks the broker to replay fixed API workflows through

judging/execute.py. Candidate snapshots and evaluator code are mounted read-only. Bounded observations, failures and errors become anonymised evidence packets. Host Python then invokes the configured adapter; the JavaScript copilot-judge.mjs establishes a separate tool-free Copilot session. The judge receives the packet and returns ordinal ratings, rationales and evidence citations for host-side validation and reporting. It has no command tools or candidate-repository access. Its model connection is outside the replay container; candidate isolation does not place the AI judge or host orchestration inside that sandbox.

The broker's candidate-execution containers disable networking, use a read-only root filesystem and set `no-new-privileges`, with CPU, memory, process and time limits. Docker uses `-cap-drop=ALL` but re-adds four for its protected supervisor; the candidate child runs as UID/GID 1001 with cleared effective, permitted and inheritable capabilities. The entire container is therefore not capability-free. Development workspace mounts remain writable; replay snapshot/evaluator mounts are read-only. Copilot authentication stays with the trusted host transport. An isolation or collection failure leaves the checkpoint unavailable. A successfully collected candidate failure remains failure evidence, rather than automatically becoming unavailable or a numerical zero rating. These boundaries describe inspected source contracts, not an adversarial integrity proof or verification of every archived execution epoch.

7.4 Outcomes and retained-contract loss

The primary outcome is final behavioural coverage: contracts passed out of all 127 scheduled feature contracts at P_{10}, before CR1. Coverage combines acquisition, retention and repair. Retained-contract loss and CR1 adaptation are distinct secondary outcomes, with legacy identifiers H2 and H6 retained only for data traceability. HIR/IDR definitions are in the supplementary appendices; comprehension is not measured.

Who measures, and what "quality" means. The running kit at `e71037d9` does not call a separate LLM-as-judge to assign numerical outcome scores. The experiment models produce features, consolidation, review notes or repairs; these are treatments, not independent measurement instruments. The host evaluator runs fixed hidden test contracts and external code-analysis tools, writes `measure.json`, and computes summaries from those records. Table 7 separates outcomes, structural proxies and resource observations. "Behavioural coverage" is a test-contract pass fraction, not instrumented line or branch coverage. Complexity, duplication and lint do not constitute a general architecture, readability or comprehension score; the active metric dictionary contains no maintainability-index score.

For fixed inputs, test oracles and static metrics are programmatic and normally repeatable. Environment-dependent code, timeouts or flaky tests can still vary. The evaluator records errors, skips and timeouts; missing or non-binary endpoints cannot silently become passing outcomes. The reported local evaluator is not a claim of OS or network isolation. Elapsed time and consumption vary with execution and provider conditions; their deterministic aggregation after recording does not make the underlying observations deterministic. Raw fields, units, scopes, tool settings and missingness are specified in Supplement Section A.8.

AI review is a treatment, not an outcome judge. Arm C's model reads reports/quality.md and the feature diff, then rewrites `REVIEW.md` in at most 300 words using a checklist of duplication, complex functions, names, dead code, responsibilities and behaviour-focused tests. Those qualitative judgements may vary between model sessions, but the kit does not convert them into a numerical review or quality score. Arm B uses a similar checklist to guide changes. Its complexity ≤ 10, duplication $< 3\%$ and zero configured-lint-warning targets are requested stopping conditions and treatment-fidelity checks. `make check` runs tests and generates the metrics report; the report writer does not itself assert that all three structural targets are met. The runner enforces the time budget, protected tests and retained green code, rather

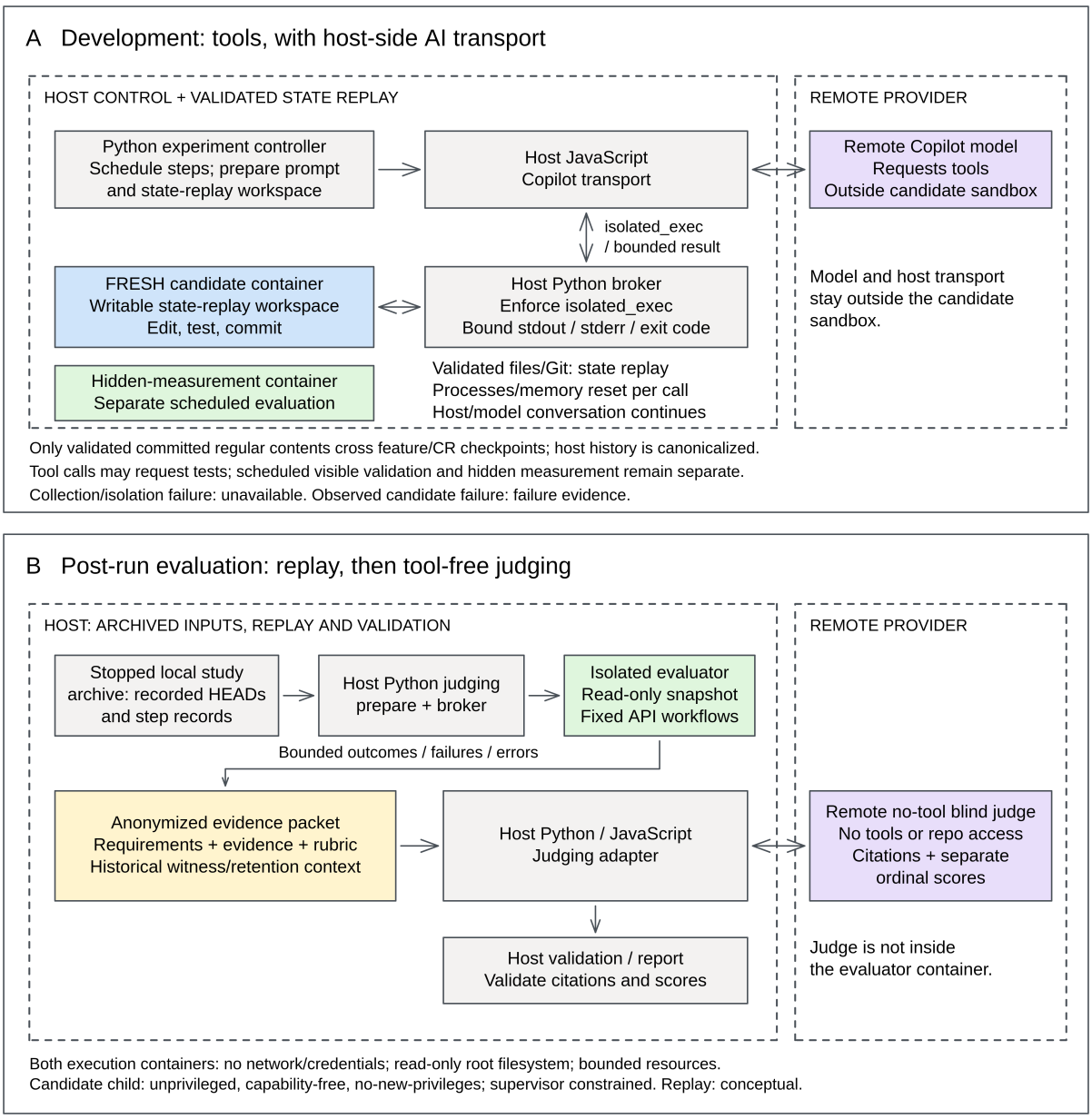


Figure 6: Source-described execution paths at `b7973e20`. Development agents request isolated commands against a writable repository; post-run judging follows read-only workflow replay and receives evidence without tools. Python orchestration and JavaScript model transport remain on the trusted host in both paths. Arrows describe information/control flow, not observed study execution or measured causal effects. Source and historical-runtime qualification are in the revision record.

than accepting an AI statement as an independent quality verdict. The exact checked-in B/C examples, parameter substitution and saved-prompt limitations are in Supplement Section A.10.

For trajectory r, F_t is the retained state after feature step t and P_t the state after its assigned post-step; these source states coincide in A and D. Let $\mathcal{R}_{r,t}$ contain eligible, unchanged contracts that passed at least one earlier end-of-cycle checkpoint P_u, $u < t$. Superseded contracts leave this risk set. At F_t and P_t, let $\mathcal{O}^p_{r,t} \subseteq \mathcal{R}_{r,t}$ contain contracts with observed binary results $S^p_{r,t}(q)$, $p \in \{F, P\}$. Report

$$L^p_{r,t} = \sum_{q \in \mathcal{O}^p_{r,t}} \mathbf{1}\{S^p_{r,t}(q) = 0\}, \qquad d^p_{r,t} = \frac{L^p_{r,t}}{|\mathcal{O}^p_{r,t}|}. \tag{1}$$

The density is checkpoint prevalence of failure among observed, previously acquired contracts.

Table 7: Measurements in the active kit inspected at `e71037d9`. Programmatic scoring is distinct from model-generated treatment text. Deterministic means repeatable for fixed source, inputs, tools and environment, not identical across different generated repositories or runtime sessions.

Measure and role	Values and instrument	Provenance
Final coverage; primary	Hidden feature contracts passed out of 127 at P_{10}, expressed on the 0–100 scale	Programmatic test oracles
Retained-contract loss; secondary	Previously acquired unchanged contracts later failing; counts, density, episodes and accumulated test-cycles	Checkpoint calculations; incomplete history can be unavailable
CR1 adaptation; secondary	All 15 CR1 contracts pass and previously passing feature contracts remain satisfied	Programmatic tests and documented disposition rules
Complexity; structural proxy	Lizard function-level cyclomatic complexity: mean, maximum and share above 10	Deterministic code analysis
Duplication; structural proxy	jscpd duplicated-line percentage, Python mode, minimum 50 tokens and five lines	Deterministic clone detection
Lint; structural proxy	Ruff diagnostic counts and warnings per 1,000 source-code lines, frozen selected rules	Deterministic diagnostics; inline suppression caveat below
Size and change; companions	Source/test code lines, files/functions, Markdown words, test/assertion counts and Git additions/deletions	Deterministic counts; not proof of code quality
Resources; cost companions	Recorded credits, input/output/cache tokens, session seconds and available API-duration telemetry	Variable observations; aggregation is programmatic, not AI judgement
Assistance and integrity; companions	Recorded help/dispositions, protected-file hashes, violations and leak indicators	Explicit records and checks; absent annotation is not zero

An empty observed denominator is undefined. These summaries require known full-risk membership. If earlier acquisition checkpoints leave membership uncertain, retain that uncertainty and the checkpoint vectors rather than treating unobserved acquisition as failure to acquire. Publish the full risk set, observed subset and missing count $M^p_{r,t} = |\mathcal{R}_{r,t}| - |\mathcal{O}^p_{r,t}|$ alongside it. The full-risk loss count lies between $L^p_{r,t}$ and $L^p_{r,t} + M^p_{r,t}$; missing evaluations cannot be treated as passes. Never-acquired contracts remain acquisition failures. Retain onset, recovery, recurrence and uncertainty across missing checkpoints. Recovery does not erase an episode. Distinguish same-cycle post-step losses and rejected attempts from losses in retained code.

The v6 analyzer inspected on 5 November uses a stricter summary rule: incomplete prior acquisition or any unobserved eligible result makes its loss count and density unavailable, with completeness flags. Its standard export does not emit the observed-subset counts, missing-count fields and bounds specified above. Those are prospective reporting requirements retained from the earlier design. The 8 November read-only inspection of active `e71037d9` confirms that the conservative rule remains in `pilot_analysis.retention`; it does not establish the planned observed-subset export or amend the reporting rule. The source-bound reporting decision remains to be documented without backdating an amendment or inferring completeness from raw vectors.

CR1 success requires all 15 CR1 contracts and retention of every feature contract passing before CR1. Also report contracts passed out of 15, previously passing contracts lost, and observed/missing denominators. Stratify by whether F9 and F10 prerequisites were ac-

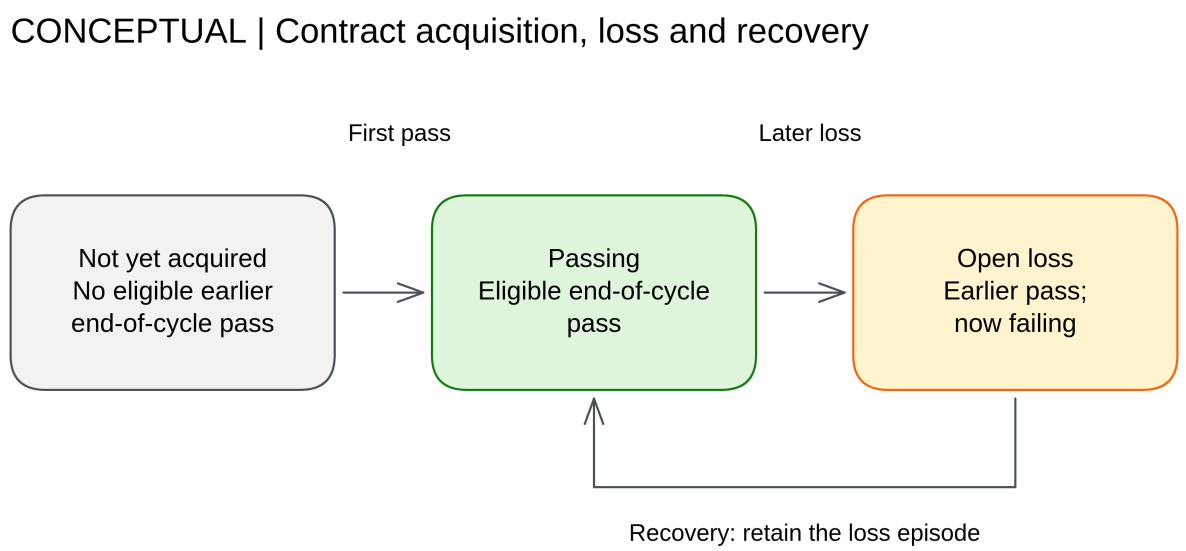


Figure 7: Conceptual contract states. Acquisition, loss, recovery and unobserved evaluation are distinct; the diagram contains no measured frequencies.

quired before CR1: an incomplete arm has fewer retention obligations, and prerequisite strata are post-treatment selections, not randomised subgroups. One fixed CR1 is task-specific. A prewritten, randomly selected request pool belongs to a separately registered subsequent study. The summary inspected on 5 November computes composite CR1 success. The active-source inspection on 8 November likewise finds composite `cr_success`, not a completed graded or F9/F10-stratified CR1 reporting pipeline. Those additional reporting requirements retain the same source-bound reconciliation gap; retained raw vectors alone do not establish a completed reporting pipeline.

Actual AI credits include every delivery and CR1 attempt, including failures, assistance and infrastructure retries; phases are reported separately. Cheap failure is not efficiency. Credits, tokens, elapsed time and currency are distinct. Lizard measures complexity, Ruff lint and jscpd duplication. The inspected Ruff invocation uses `-isolated` with the fixed `E,F,W,C90,B,SIM` rule selection, ignoring repository configuration files, but does not include `-ignore-noqa`. Inline `# noqa` can therefore suppress diagnostics, and the active metric dictionary exports no separate suppression count. The earlier description overstated this safeguard. This is a measurement-integrity limitation, not evidence that participants used suppression. This manuscript clarification changes no running instrumentation; any subsequent fix needs explicit source/epoch and amendment disclosure. SDK request wall time and tool time are observable. Provider-active time and rate-limit waits require explicit telemetry; absent telemetry is not zero.

7.5 Supplementary post-run workflow assessment

A separate judging enhancement proposes an exploratory assessment of the software inherited at the end of development. After collection stops, a fixed specification-led panel will replay API workflows against archived S1 snapshots, without rerunning development agents or repairing their repositories. The source inspected on 9 November 2026 is `b7973e20`; Supplement Section A.9 records its full pin and measurement rules. This assessment leaves the running protocol, treatment budgets and original coverage, retained-contract-loss and CR1 endpoints unchanged. Its post-collection design supplies supplementary exploratory evidence, with no

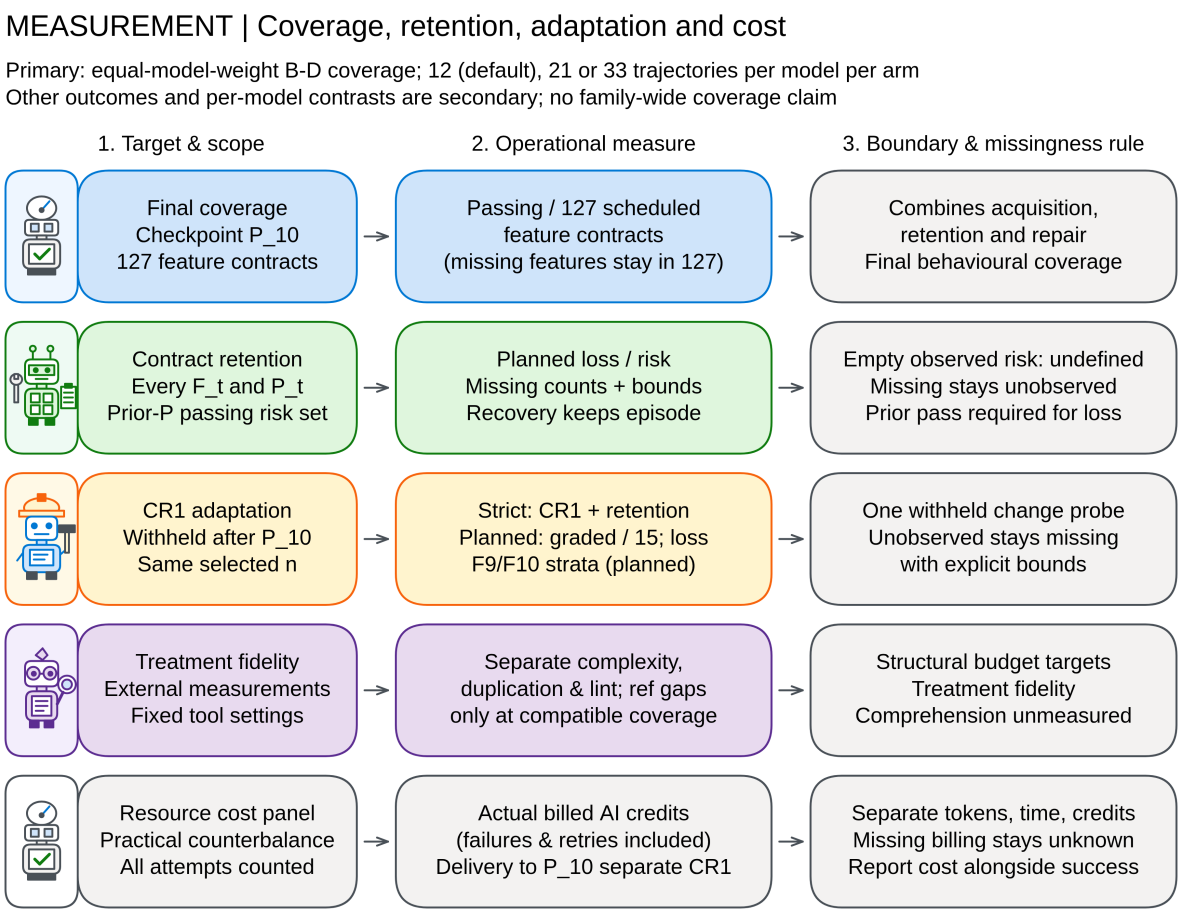


Figure 8: Historical measurement design: coverage, retained behaviour, adaptation and all-attempt cost are separate measures. Structural metrics targeted by B's stopping budget are treatment-fidelity checks, not independent efficacy outcomes. Unknown results and billing remain unknown. The observed-subset retention and graded CR1 panels describe planned reporting requirements; their implementation was unresolved in the 5 November v6 inspection. The active-source inspection confirms conservative loss and composite CR1 summaries, not those planned richer exports.

preregistration or confirmatory status inferred from freezing the judging instrument.

The panel covers ten feature workflows, a cross-feature delivery/reopening journey and a separate CR1 workflow. Each scenario becomes eligible when its dependencies have been scheduled. Recorded end-of-cycle HEADs supply P_1–P_{10}, with P_{10} after the applicable consolidation, review or repair step; post-CR1 is assessed separately. A failing endpoint is classified as witnessed capability loss only when that scenario passed at an earlier eligible checkpoint. Acquisition failures and unavailable observations remain distinct. These workflow witnesses complement the more extensive hidden-contract instrument. Post-CR1 endpoint loss can include a failure already present at P_{10}; it does not isolate loss caused by CR1.

Separately configured AI judges will receive anonymised execution evidence and rate functional completeness, workflow coherence, error recovery and API usability on four distinct ordinal 0–4 scales, citing observations and abstaining where evidence is insufficient. These ratings address the exercised API tasks; human interface usability and satisfaction require different evidence. The inspected packets also expose earlier passing witnesses and retention labels, which can anchor nominally endpoint-focused ratings; this contextual limitation requires disclosure. Arm C's in-development review remains a treatment, separate from this post-run assessment.

Repeated judge sessions will describe rating variability within a trajectory. Arm contrasts require the complete allocated comparison frame and equal weighting of within-development-

model differences. Trajectories remain the study units; scenarios, snapshots and repeated ratings do not increase independent n. Human calibration, residual blinding risks, source epochs and scheduling dependence require explicit reporting before interpretation. Judging costs are accounted for separately.

7.6 Replication and uncertainty

The 2 November planning record offers $n \in \{12, 21, 33\}$ independent repository trajectories per model per arm, with **12 as the default**; current execution selects 21. That record calls for the choice, rationale, planning SD, power/precision trade-off, date and source-bound configuration to be recorded before collection. It does not establish that the current amendments were prospectively registered. Selecting a sample size does not establish that its associated SD assumption is true. The saved allocation is immutable on resume. Outcome-driven sample-size changes, additions and replacements are prohibited; interim coverage, loss, CR1 success or other outcomes cannot determine n.

Servera Bondroit selected a minimum useful absolute coverage gain of ten percentage points, approximately thirteen test-weighted contracts. Coverage scores Y are expressed on the 0–100 scale. Let $\overline{Y}_{ma}$ be mean final coverage for model m and arm a. The primary estimand is the equal-model-weight difference in mean coverage over the $M = 3$ selected models. Its estimator is

$$\widehat{\Delta}_{BD} = \frac{1}{M} \sum_{m=1}^{M} (\overline{Y}_{mB} - \overline{Y}_{mD}). \tag{2}$$

For balanced cells of n_{cell} independent trajectories and a common within-cell variance σ^2, its planning variance is $2\sigma^2/(Mn_{\text{cell}})$, with $n_{\text{cell}} = n$. For one two-sided comparison with $\alpha = 0.05$, target power $1 - \beta = 0.80$ and target difference $\Delta_* = 10$ points, the normal planning minimum, conditional on the assumed SD, is

$$n_{\text{plan}}(\sigma) = \left\lceil \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{M\Delta_*^2} \right\rceil. \tag{3}$$

Table 8: Historical conditional normal planning for the ten-point B–D contrast, three equally weighted models, $\alpha = 0.05$ and target power 0.80. The SDs are assumptions, not measured variances or interchangeable power guarantees.

Assumed within-cell SD (points)	Planning minimum per model per arm
15	**12 (default allocation)**
20	21
25	33

These are conditional planning assumptions, not estimates of actual within-cell variability. Target power depends on independent trajectories, the common variance, normal approximation and equal-model estimand; it is not guaranteed by any offered choice. In particular, default $n = 12$ is *not* an 80%-power design under the existing SD of 25 points. The bounded score's worst-case SD is 50, requiring 131 per cell under the same approximation; that is outside the offered allocation choices. These planning requirements and assumptions are retained from the earlier record. Neither the registered base nor selection of $n = 21$ establishes their validity for evolving-source exploratory data.

The candidate independent-cell analysis resamples whole trajectories within model and arm, with 10,000 draws and pointwise 95% percentile intervals. The inspected v6 code uses independent arm/model draws; it does not preserve joint scheduling-block draws. The inspected

primary requires a complete allocated five-arm/model metadata frame, known unassisted status and observed contributing B/D final coverage. When that frame is unmet it reports the primary unavailable. Retain the allocated frame and costs; secondary outputs cannot replace it with an observed-model pooled or complete-case primary. An earlier missing H2 checkpoint can coexist with an observed final coverage or CR1 endpoint. This availability rule also needs explicit adoption or prospective revision in the approved study record. Model-specific estimates and other contrasts are secondary, with no family-wide coverage claim. Retain scheduling-block identifiers. The earlier source-bound review identified an unresolved choice between this independent-trajectory variance/bootstrap scheme and model/replicate blocking. A scheduling label alone establishes neither pairing nor independence. Scheduling-only blocks with independent inference require justification; block-aware analysis would require a separately approved, disclosed choice and aligned planning variance. Shared block/service effects remain unmeasured; the v6 source freeze does not resolve this choice, and this update supplies no new resolution. Chen, Zaharia and Zou's dated ChatGPT comparison motivates monitoring service drift despite exposed names [22]; it does not measure drift in this roster or make a version pin an immutability guarantee. Report the pooled within-cell SD with its degrees of freedom for future planning. For binary CR1, report per-model success counts and exact binomial component intervals under independent runs with a common success probability within each cell. Aggregate contrasts must preserve model strata. The coverage calculation supplies no power guarantee for CR1.

7.7 Historical resource planning and operational boundaries

The following resource scenarios and feasibility criteria retain the 2–5 November planning record. In particular, their full ten-feature calibration differs from the short calibration disclosed for ongoing unregistered execution. They are not measured costs, a current affordability approval or evidence that historical launch gates were satisfied.

The actual aggregate envelope remains 300,000 AI credits, with a 270,000-credit local admission ceiling. Prior spending and provisions for unknown historical usage remain charged or held against remaining admission; unknown usage is never zero or free headroom. Original 300-credit sessions use a 270-credit soft limit; D's 600-credit feature uses 540. A retry shares the original allowance. In-flight responses can overshoot; these are not provider billing caps. No verified dollar conversion is available.

Each study trajectory contains ten feature cycles and one CR1 session, the latter capped at 45 minutes and 300 nominal credits in every arm. A/D therefore have eleven sessions per trajectory; B/C/E have twenty-one. A allows at most 5.75 agent-hours and 3,300 nominal credits per study trajectory; B/C/D/E each allow 8.25 hours and 6,300 credits. Across three models and five arms, the study has $15n$ trajectories, $3n[2(11) + 3(21)] = 255n$ sessions, $3n[5.75 + 4(8.25)] = 116.25n$ maximum agent-hours and $3n[3300 + 4(6300)] = 85500n$ nominal session credits.

The historical excluded cost-only calibration scenario retains one trajectory per model per arm: 15 trajectories, ten features each and no CR1. A/D have ten sessions and B/C/E twenty; A allows five hours and 3,000 credits, and B/C/D/E each allow 7.5 hours and 6,000 credits. Thus calibration adds 240 sessions, 105 maximum agent-hours and 81,000 nominal credits, independent of n.

Combined nominal allowances are $85500n + 81000$ credits and maximum agent-hours are $116.25n + 105$. At default $n = 12$, 1,107,000 nominal credits and 3,300 sessions including calibration do not authorize spending beyond the unchanged 300,000-credit envelope or 270,000-credit admission ceiling. Excluded calibration informs cost and time feasibility only, not outcome- or variance-driven sample resizing. Its cost forecast must show that the entire selected allocation, including CR1 and all-attempt liabilities, fits remaining admission and the deadline after prior spend and unknown-usage holds. Otherwise halt before study collection. A different allocation requires a separately authorized prospective amendment and a new saved study, never a changed allocation on resume or a silent reduction to fit the budget. An interrupted or unaffordable

Table 9: Historical five-arm resource scenarios for the offered choices of n per model per arm; the historical default column is bold. These are summed nominal session allowances and time ceilings, not expenditure or enlarged budgets. Every choice exceeds the aggregate envelope at nominal session caps.

Quantity	$n = 12$ **(default)**	$n = 21$	$n = 33$
Study trajectories per arm, all models	**36**	63	99
Study trajectories, all arms	**180**	315	495
Study sessions, including CR1	**3,060**	5,355	8,415
Study maximum agent-hours	**1,395.00**	2,441.25	3,836.25
Study nominal session credits	**1,026,000**	1,795,500	2,821,500
Excluded calibration trajectories, no CR1	**15**	15	15
Excluded calibration sessions	**240**	240	240
Calibration maximum agent-hours	**105.00**	105.00	105.00
Calibration nominal session credits	**81,000**	81,000	81,000
Combined trajectories	**195**	330	510
Combined sessions	**3,300**	5,595	8,655
Combined maximum agent-hours	**1,500.00**	2,546.25	3,941.25
Combined nominal session credits	**1,107,000**	1,876,500	2,902,500

study retains its selected allocation and is reported incomplete. The launcher inspected on 5 November enforces attempt and aggregate admission but does not compute or approve that full-allocation forecast. A dated researcher forecast and approval are required separately; a nonempty calibration note is insufficient. Proposed progression criteria are infrastructure failures at most 5%, no missing primary observations, complete billing/model/help records, zero oracle contradictions in triplicate reference checks, and no information-boundary failure. Hidden-test mutation assessment must report its actual mutant denominator and unresolved equivalence. These are feasibility criteria, inspired by pilot-study guidance [30], not a clinical CONSORT compliance claim.

Only current and previous visible specifications and examples reach candidate tools. Hidden tests, scores, identifiers, messages, reference code and future CR1 remain external. Candidate commands use no-network containers; the trusted host SDK's provider connection is different. The earlier plan called for a researcher-managed protected workspace without the original case repositories, manuscript or session-context caches. Cabezas Martin's current execution role is separate from design and manuscript preparation; coauthorship supplies no evidence of blindness or actual access restrictions. Record actual access and prior exposure. The earlier repository-privacy and later approved-release plans remain historical intentions, not inspected current visibility. OSF status and GitHub privacy are separate; neither removes prior public or model exposure. The historical registered-freeze path and current disclosed unregistered path have different registration status. This manuscript update neither verifies operational approvals nor changes information boundaries, budgets, configuration or launch permissions.

Unsuccessful feature increments are reset; later features and scheduled B/C/E post-steps continue on retained green state. Hidden denominators still include undelivered features. One infrastructure-only retry shares the original budget; poor performance never justifies replay. Contradictory valid oracle results halt collection. Assisted and incomplete trajectories and all their costs are retained, without replacements or success-conditioned primary analyses.

8 Discussion

8.1 Projected cost of omitting consolidation

The experiment measures costs over ten features and one final change. It does not yet estimate the price of maintaining a larger system without refactoring. Figure 9 and Table 10 therefore give an explanatory sensitivity scenario, not results, an empirical forecast or an addition to the frozen analysis. The economic hypothesis is that early investment can reduce the implementation and repair burden inherited by later features. It can fail: refactoring may consume credits without reducing that burden or may damage behaviour. Cost must always be read beside coverage, retention and adaptation.

Let $N \geq 1$ be an integer number of feature cycles, k the assumed average consolidation investment in AI credits per cycle, and $\delta \cdot (t-1)$ the assumed excess feature-implementation cost in A relative to B at cycle t, excluding B's consolidation session. Shared baseline implementation costs cancel. Under the explicit simplifying assumptions of constant k, a linearly growing relative premium $\delta \geq 0$, and comparable delivered behaviour, the cumulative net difference is

$$\Delta C(N) = C_A(N) - C_B(N) = \sum_{t=1}^{N} [\delta(t-1) - k] = \delta \frac{N(N-1)}{2} - kN. \tag{4}$$

The first term is the assumed gross avoidable burden of omitting consolidation; the second is the investment to avoid it. A positive difference means A costs more under these assumptions, not measured savings. For $k > 0$ and $\delta > 0$, the first cycle count with $\Delta C(N) \geq 0$ is $N_{\mathrm{BE}} = \lceil 1 + 2k/\delta \rceil$; when $\delta = 0$, the investment never pays back in this credit-only model. Real trajectories need not follow a linear premium, and B can retain debt too: δ is a relative gap, not A's absolute debt or a claim that B is debt-free.

Table 10: Illustrative net A–B cost in AI credits from Equation 4, with assumed $k = 5$ credits/cycle. Negative values mean B's investment has not paid back; zero and positive values mean break-even and a conditional credit advantage. None of the scenarios is an observed or preferred estimate.

Assumed premium increase δ	10 cycles	20 cycles	40 cycles
0 credits per successive cycle	-50	-100	-200
0.5 credits per successive cycle	-27.5	-5	190
1 credit per successive cycle	-5	90	580

The study can report actual B consolidation consumption and all-attempt CR1 cost by model. Calibration can anchor k once billed spend exists. Because feature budgets are capped, harder work can instead reduce coverage: δ is an uncapped, quality-matched counterfactual, not a directly identified endpoint of this design. Even the enlarged replication over ten features does not identify a stable long-run δ or justify extrapolation to 40 features. A future forecast would require longer replicated sequences, matched delivered quality, measured residual repair work and independently checked billing. Human review, operational failures, opportunity costs and currency prices are not measured by this kit and cannot be monetized from its credits or code-size proxies. Thus the current projected cost of inaction is conditional and unestimated, not a claimed business saving. For comparison, assumed $k = 100$ and $\delta = 1$ yield break-even at 201 cycles, not eleven. No observed consolidation spend supports a preferred k yet.

8.2 What worked

Agents contributed useful challenge and rapid iteration. The research phase assembled a corpus and analysis in days; a human-only duration was not measured. The role agents' challenge killed a bad idea, five end-to-end agents, before any code was written. The agents refused

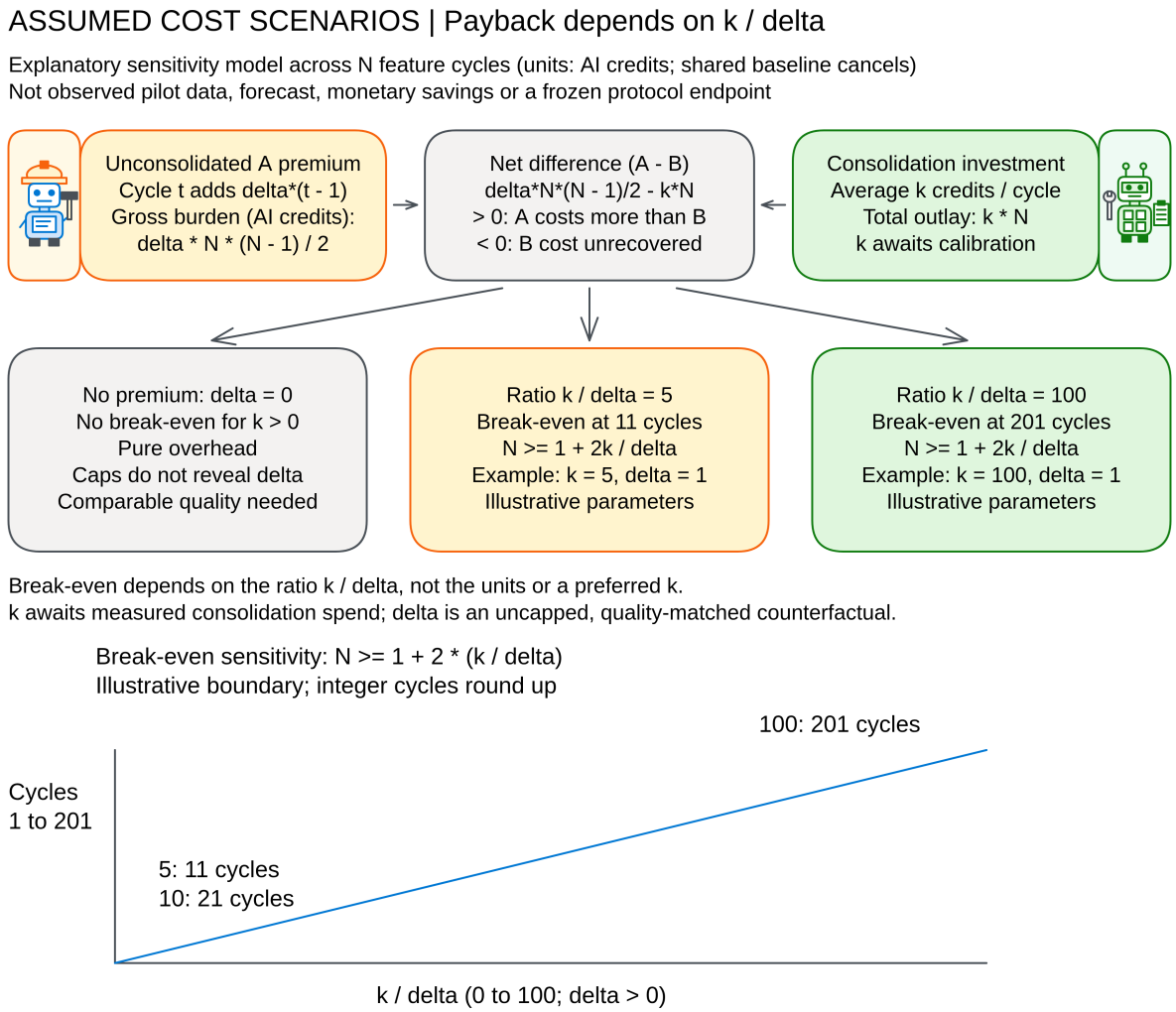


Figure 9: Assumed cost-of-inaction mechanism and break-even sensitivity, not pilot results or an empirical projection. A linearly increasing relative implementation premium accumulates across feature boundaries; recurring consolidation has an explicit cost. Payback depends on k/δ: a ratio of 5 breaks even at 11 cycles and a ratio of 100 at 201; $\delta = 0$ gives no payback for $k > 0$. Parameters are invented for transparent sensitivity, not fitted to study data. No dollar conversion or human-work estimate is implied.

to invent approvals, figures or company facts, and recorded the limits of their evidence. The MVP was built and reshaped in a day. These episodes included targeted human challenge and feedback; they are not a controlled comparison with human-only teams or proof that all short loops succeed.

8.3 Human judgement and reflective work

The participant supplied several kinds of reflective work: choosing what not to build, recognising the unusable replacement and deciding which earlier lessons mattered. Naur's theory-building account helps describe that retained understanding [65]. Beck's distinction between visible output and understanding supplies another framing [14]. These judgements were supplied by the author during the case. Whether agents could make comparable judgements remains open. The narrower experimental question is whether bounded post-feature correction preserves behaviour and later changeability.

Human sign-off also need not establish who formed a judgment. In Anthropic's randomised study, 61 analysts were recruited and 50 used the agent. Those 50 analysts matched their agent's

randomly assigned preference on 86% of four analytical choices [39, Study design; Results]. The preferences were deliberately strengthened in one farming-analysis task with one model; the rate concerns a different task and setting from coding [39, Our study's limitations]. This challenges an assumption that retaining approval alone preserves independent judgment, not the case's reported decisions. The article proposes asking experts for their view first or making the best case for each option [39, What this means]. Separately, recording alternatives and a human's reasons before accepting an agent's recommendation is the present author's proposed safeguard, not a remedy evaluated in that study.

8.4 Can another agent restore futures?

Beck's alternation suggests preserving the options consumed by feature execution [12, 15]. Darwin, AutoSaddler and SkillOpt already provide precedents for agent-guided correction; reflection itself is not the proposed novelty. Ng's reflection framework also argues for evaluating and revising work rather than treating first-pass generation as the finished product [66]. Improvement here means a measured advantage in retained behaviour or later adaptation, not merely writing a lesson, changing a prompt or calling the process self-improving. The experiment does not train model weights; the evolving repository and handover are the states under test. An exposed model name is not a guarantee against provider-side changes.

The memory presentation's proposed "dreaming" pass is a useful contrast: it curates prior transcripts into persistent context [5], whereas this study changes retained code, tests and the handover. Memory updates can carry an incorrect lesson forward just as code updates can retain a defect. A self-improvement label therefore cannot replace the retention and adaptation measures. The study asks what remains after a fixed repository-consolidation step, with CR1 as one future-change probe. It does not test choosing when to interrupt delivery, changing product goals or restoring a deployment. Those questions, and matched human/agent steering, belong to the separate research agenda in the ancillary supplement.

8.5 Implications for practice

The case motivates protecting a usable increment, bounding the delivery horizon and keeping lessons retrievable or executable. The ancillary supplement retains the proposed practice checklist and framework-builder suggestions. For future product integrations, routine intermediate tracking files should stay out of the squash-merged product diff once durable findings and original review evidence have been preserved. This is a proposed practice, motivated by the large intermediate evidence trails retained in this case. It is not permission to wipe an active or historical tracking tree: selection must be finite, archive-backed and restore-verified, with adverse reviews retained. Proportionality never overrides mandatory safety, privacy, approval or publication gates, and compact context views must retain their source evidence. The pilot does not test that practice package, an optimal human-intervention cadence or the preservation of expertise. Those remain separate questions.

9 Conclusion

The unit of progress in cumulative development is the system left for the next change. Knight Capital shows how a new deployment can activate an old fault. WyCash makes consolidation part of continued growth. Agent research adds a direct cross-feature test: in SWE-Maintain, earlier changes affect later tasks even when conversation history is discarded. Its clean-history results make preserving behaviour a more precise target than treating all agent-written code as structurally deficient.

The instructions governing development need maintenance too. Repository studies show repeated additions and revisions, alongside pruning; controlled context-file comparisons show

that more guidance can increase work without improving task resolution. Code, tests and instructions are all inherited state, but they require different measures of correctness and burden.

The motivating case traces a fuller process: agent-assisted research and challenge shaped requirements, prototypes and a hierarchical backlog; autonomous issue delivery then failed to preserve the participant's usable workflow. Renewed feedback, refinement and refactoring redirected subsequent work. The five candidate mechanisms frame possible links among accumulated obligations, lost lessons, completion proxies and insufficient product-use feedback. Migration and deployment dependencies remain central to that particular failure. The versioned repository retains the evolving record and the distinction between local engineering evidence and accepted journeys (Section 4.4).

The ongoing five-arm exploratory study asks what scheduled consolidation leaves for later work. It separates final behavioural coverage from retained-contract loss, probes an unforeseen change and reports the resources consumed. Secondary metrics provide descriptive evidence; testing the proposed mechanisms would require further work. Collection follows an owner-confirmed OSF-registered base with disclosed unregistered amendments, including changes to calibration and execution. The retained record must distinguish implementation epochs and their potential temporal, arm and model confounding (Section 7.1). The cost-of-inaction scenario makes the break-even assumptions explicit. What matters is whether the agents' combined work preserves the behaviour and options needed by subsequent features.

Acknowledgements

The authors thank the colleagues and partners whose questions shaped this report.

Author contributions

Juan Manuel Servera Bondroit conceived the study, designed the methodology, defined the evaluation criteria, analysed the historical exploratory case and wrote the manuscript. Experimental analysis remains his responsibility as collection proceeds. Isabel Cabezas Martin is conducting the ongoing experiment and collecting its data independently under the predefined exploratory protocol and disclosed amendments. Separating execution and data collection from study conception, design and manuscript preparation is intended to minimise potential experimenter bias. Both authors are Microsoft employees: this operational separation establishes neither institutional independence nor independent scientific review or analysis, and does not demonstrate elimination of bias.

Use of generative AI

GitHub Copilot with HVE Core agents supported development of the system under study. The case method distinguishes reported interface labels from recorded selectors; the deployed assistant exercise used `gpt-4.1-mini-2025-04-14`, not the coding-agent model. Microsoft 365 Copilot supported summarisation and reference discovery.

GitHub Copilot in VS Code assisted log analysis, mechanism naming and organisation, manuscript drafting and revision, bibliographic checks, public repository inspection, figure preparation and local validation. Scientific Agent Skills supplied developmental writing and methodology guidance, including the SymPy, statistical-analysis, scientific-critical-thinking and experimental-design skills for this equation review [45]. This citation records assistance tooling, not evidence for the case findings or consolidation hypothesis. AI assistance supplied neither an independent empirical coder nor replicated application or study results. The package README retains the revision-by-revision account and source-access limits. GitHub Copilot

CLI assisted the 29 October source integration and scoped method-clarity revision. For the subsequent editorial and visual revision, Gemini 4 Argon proposed infographic concepts. Those suggestions were checked against the manuscript and adapted into editable Excalidraw conceptual diagrams; they are not experimental outputs or independent validation. The earlier 1 November v8 revision reconciled the experiment description with v4, added the measurement and conditional cost-of-inaction explanation, and used Gemini 4 Argon for editable infographic design and implementation. AI-generated conceptual graphics and assumed economics are explicitly separate from experimental data; human author approval remains required. The later v9 revision used Copilot-assisted source comparison, static measurements, five-arm kit implementation and local native diagram authoring, as reported in its dated record. The 2 November allocation record assessed A/B/C-only capability from the then-inspected snapshots. A 5 November source check superseded that assessment: the then-inspected v6 source contains all five arms and the immutable allocation selector. The recovery used Copilot-assisted source review, symbolic and numerical checks, authoring regressions and PDF inspection. It added no experimental trajectories, independent human peer review or new Gemini design-session claim. The 8 November revision used Copilot-assisted editing and the scientific-critical-thinking skill to qualify operational separation, registration evidence and implementation provenance. It inspected the supplied commit-bound public protocol and relevant public README and contributor passages, and used the existing authoring checks and PDF tooling. The measurement clarification inspected the public measurement engine, collector, pilot analyzer, usage parsing and B/C prompt templates. It added instrument/field tables, verbatim source-bound prompt examples and the Ruff inline-suppression limitation, without changing experimental scoring or instrumentation. It performed no model runs or experimental outcome analysis. The 9 November revision used Copilot-assisted inspection of the supplementary judging enhancement, a separate agent context guided by the supplied Data Science and Engineering Coach definition, and the scientific methodology skills cited above. The launcher exposed a general-purpose agent rather than the named Coach. This assisted assessment informed prospective workflow, rating and missingness descriptions; it supplied no experimental judgments, human calibration or independent scientific acceptance.

Servera Bondroit supplied the case observations, selected the case and made the analytical judgements, and reports checking the repository figures and Azure expenditure and confirming the Beck paraphrase after reading the essay and watching the talk. The supplied automatic critique and this revision's focused source and arithmetic checks are assisted evidence assessments, not independent human verification. The package records their coverage and unavailable sources. Metadata checks do not establish claim support; source comparison does not independently validate a source's experiments. The human authors retain responsibility for the content and final approval. Further disclosure must follow the actual submission venue's policy, which has not been assessed here.

Competing interests

Both authors are Microsoft employees. The study concerns Microsoft and GitHub tools, and its historical case arose within a Microsoft training programme. Servera Bondroit designed the coordinator and analysed a case in which he was a participant; Cabezas Martin conducts experimental execution and data collection separately from those roles. These professional and participant-observer interests remain relevant despite the operational separation. The authors' views do not imply company endorsement. Employer external-publication clearance and final-author approval remain separate responsibilities.

Data availability

The repository analysed was publicly accessible during the case at https://github.com/jmservera/ama; this report uses commit `29c6b512f5e68733bb2cc1882343db39f5eb2cad`. The separate 29 October repository follow-up pins `9b74ac8daaec2a880172e43161b37ac7f2c3f801`; its selected API records, retrieval interval, merge-window definitions, document hashes and metrics are in anc/repository-metrics-2026-10-29.json. The log-measurement and figure-generation scripts are provided with the source. The ancillary file anc/case-evidence-matrix.json indexes O1–O5, including reported locators and missing release/execution links; it is not newly collected corroboration. The companion anc/supplementary-context.md retains practitioner material, external observations, proposed practices and deferred extensions. The ancillary file anc/repository-observations-2026-10-25.json records the additional repositories' source revisions, tracker counts, sampling boundary and claim locators without reporter identities or attachments. Live issue states can change independently of the pinned source commits. The ancillary experiment notes in HVE Core's Minimum Viable Experiment format are earlier proposals, not the tagged study record. The separate researcher kit is at https://github.com/jmservera/ama-thought-mve. Preserve its historical frozen v4 release alongside the historical source-published v6 release. The 5 November read-only check inspected default commit 5ab24be4556876500a40e56201f6efc3930cc8b8, a manifest-only child of v6's execution source 7de2f0d42955c62b52906747de480df2b3f1e0ca. That source implements A/B/C/D/E and a saved, immutable `-replicates` selection of 12, 21 or 33, default 12. That dated source inspection verified the implementation and release identities; runtime-image qualification and source-declared test totals were not reproduced. The then-inspected v6 manifest had no registration date, recorded no prospective OSF update and left collection unauthorized. These are 5 November findings, not current OSF metadata or a freeze of the ongoing run. Historical v4, v6 and v7 records remain distinct from current execution.

The active source is commit e71037d961cb80fc5c659437796de7736820f6c9, tree e7ec1561d4ecee921e7c9eba3711a1ab7e9f4058, without a tag. The inspected public protocol/pilot-estimation.json specifies `model-stratified-v2-five-arm`, S1, ten features, CR1 and no formal hypotheses. Its file default is 12; the owner-supplied runtime selection is 21 trajectories per model per arm, or 315 planned trajectories. The source's false/null current-registration object concerns the uncompleted amendment, rather than absence of the base study.

The owner confirms the registered base at https://osf.io/xcpt4, dated 7 November 2026. Supplied access checks returned only an OSF app shell and HTTP 401 from https://api.osf.io/v2/registrations/xcpt4/; the registration identity and date remain owner-confirmed rather than independently verified OSF metadata. The current run includes disclosed unregistered amendments for short calibration, parallel execution and VM restarts. The pinned public README and contributor instructions disclose these departures from the earlier registration. Public disclosure does not make the amended run a single preregistered confirmatory dataset.

Table 11: Implementation epochs and owner-reported completed steps as of 8 November 2026. Steps can include nonpasses; they are neither independent replicates nor completed trajectories. Source identifiers describe implementations, not collection times.

Epoch	Execution source and correction	Reported steps
E1	99a1ed170d7c71acb428b52e0e24be8856cc3ea3; pinned CLI fix	651
E2	3c2ce89b00fe583c1f0715e8ccd3510419d5e734; source-bound estimation recovery	227
E3	e71037d961cb80fc5c659437796de7736820f6c9; failed-request charging fix	369

The owner-reported total is 1,247 completed steps. This revision did not inspect the private

dataset, collection timestamps, completed-trajectory counts or reconciled costs. Epoch labels, failures, unknowns and actual arm/model/time assignments must remain available for analysis; temporal, arm and model confounding limits homogeneous pooling. No post-hoc sensitivity result is asserted. Exact commit metadata, supplied execution status and source-access limits are retained in anc/revision-report.md. The earlier protected-workspace and post-collection-release plans remain historical intentions; current access, prior exposure and release permissions require their own records. OSF status and GitHub visibility are distinct. This paper revision changes neither experiment sources, configuration, registration, repository visibility nor collection. Sanitized pilot data and exact execution provenance still require a later authorized deposit; no completed independent replication is claimed.

References

[1] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux. Spec-driven development as prescribed and as practiced. arXiv:2608.25202, 2026.

[2] L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi:10.1016/0005-1098(83)90046-8.

[3] S. Baltes et al. Guidelines for empirical studies in software engineering involving large language models. *Empirical Software Engineering*, in press; arXiv:2508.15503v8, 2026.

[4] G. Bavota, A. De Lucia, M. Di Penta, R. Oliveto, and F. Palomba. An experimental investigation on the innate relationship between quality and refactoring. *Journal of Systems and Software*, 107:1–14, 2015. doi:10.1016/j.jss.2015.05.024.

[5] beamnxw (uploader). 99% Use Agents Wrong: Anthropic's Secret: Self-Improving Agents (FREE GUIDE). Third-party YouTube upload, 18 October 2026; automatic English captions inspected 1 November 2026. The speaker identifies herself as Lamis, Anthropic Applied AI. Original recording date and audio wording unverified; headline percentages are not findings. https://www.youtube.com/watch?v=0CG2YlJZWVU.

[6] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas. Manifesto for agile software development, 2001. https://agilemanifesto.org/. Accessed 1 November 2026; manifesto excerpt.

[7] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[8] K. Beck and C. Andres. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 2nd edition, 2004.

[9] K. Beck. `test && commit || revert`. Medium, 28 September 2018. https://medium.com/@kentbeck_7670/test-commit-revert-870bbd756864. Retrieval attempted 1 November 2026; HTTP 403, full text unavailable.

[10] K. Beck. *Tidy First? A Personal Exercise in Empirical Software Design*. O'Reilly Media, 2023.

[11] K. Beck. Augmented coding: Beyond the vibes. *Software Design: Tidy First?* (newsletter), 25 June 2025. https://tidyfirst.substack.com/p/augmented-coding-beyond-the-vibes. Accessed 1 November 2026; brief excerpt.

[12] K. Beck. Why does development slow? *Software Design: Tidy First?*, 19 November 2025. Author's summary at https://kentbeck.com/summaries/why-does-development-slow/. Accessed 1 November 2026; author's complete summary, not the full essay.

[13] K. Beck, interviewed by G. Orosz. TDD, AI agents and coding with Kent Beck. *The Pragmatic Engineer* podcast, 11 June 2025. https://www.youtube.com/watch?v=aSXaxOdVtAQ. Video-page retrieval attempted 1 November 2026; HTML only, video/transcript not inspected.

[14] K. Beck. Mathematicians, here's a way to think about your existential crisis. *Software Design: Tidy First?* (newsletter), 29 September 2026. https://newsletter.kentbeck.com/p/mathematicians-heres-a-way-to-think. Accessed 1 November 2026; brief excerpt.

[15] K. Beck. Software engineering in the age of AI. Talk at Prodacity 2026, Nashville, TN, 25–27 August 2026. Video: https://www.youtube.com/watch?v=F8fBgDCf2Y4; transcript: https://www.rise8.us/videos/kent-beck-software-engineering-in-the-age-of-ai. Transcript accessed 1 November 2026; video-page HTML only, video not inspected.

[16] T. Besker, A. Martini, and J. Bosch. Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work. In *Proceedings of the International Conference on Technical Debt (TechDebt '18)*, pages 105–114. ACM, 2018. doi:10.1145/3194164.3194178.

[17] C. Bird, V.-P. Ranganath, T. Zimmermann, N. Nagappan, and A. Zeller. Extrinsic influence factors in software reliability: A study of 200,000 Windows machines. In *ICSE Companion*, pages 205–214. ACM, 2014. https://www.microsoft.com/en-us/research/publication/extrinsic-influence-factors-in-software-reliability-a-study-of-200000-windows-machines/. Accessed 1 November 2026; publisher abstract and ACM/Crossref metadata.

[18] B. Böckeler. Understanding spec-driven-development: Kiro, spec-kit, and Tessl. martinfowler.com, 15 October 2025. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html. Accessed 1 November 2026; brief excerpt.

[19] M. Borg, D. Hewett, N. Hagatulah, N. Couderc, E. Söderberg, D. Graham, U. Kini, and D. Farley. Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability. *Empirical Software Engineering*, 31(6):161, 2026. doi:10.1007/s10664-026-10889-1.

[20] F. P. Brooks, Jr. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. doi:10.1109/MC.1987.1663532.

[21] W. Chatlatanagulchai, H. Li, Y. Kashiwa, B. Reid, K. Thonglek, P. Leelaprute, A. Rungsawang, B. Manaskasemsak, B. Adams, A. E. Hassan, and H. Iida. Agent READMEs: An empirical study of context files for agentic coding. *ACM Transactions on Software Engineering and Methodology*, 2026. doi:10.1145/3840295. Inspected text: arXiv:2511.12884v2. https://arxiv.org/abs/2511.12884v2.

[22] L. Chen, M. Zaharia, and J. Zou. How is ChatGPT's behavior changing over time? arXiv:2307.09009v3, 2023. https://arxiv.org/abs/2307.09009v3.

[23] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. arXiv:2603.03823, 2026.

[24] A. Cockburn. *Crystal Clear: A Human-Powered Methodology for Small Teams.* Addison-Wesley, 2004.

[25] J. Cohen. *Best Kept Secrets of Peer Code Review.* Smart Bear Inc., 2006.

[26] W. Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings of OOPSLA '92 (Experience Report)*, pages 29–30. ACM, 1992. doi:10.1145/157709.157715.

[27] D. Delimarsky. Spec-driven development with AI: Get started with a new open source toolkit. The GitHub Blog, 2 September 2025. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/. Accessed 1 November 2026; brief excerpt.

[28] P. Devanbu, T. Zimmermann, and C. Bird. Belief & evidence in empirical software engineering. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 2016. doi:10.1145/2884781.2884812.

[29] S. G. Eick, T. L. Graves, A. F. Karr, J. S. Marron, and A. Mockus. Does code decay? Assessing the evidence from change management data. *IEEE Transactions on Software Engineering*, 27(1):1–12, 2001. doi:10.1109/32.895984.

[30] S. M. Eldridge, C. L. Chan, M. J. Campbell, C. M. Bond, S. Hopewell, L. Thabane, and G. A. Lancaster. CONSORT 2010 statement: extension to randomised pilot and feasibility trials. *BMJ*, 355:i5239, 2016. doi:10.1136/bmj.i5239. Accessed 1 November 2026; methods/reporting excerpt.

[31] M. Fowler. Design stamina hypothesis. martinfowler.com, 20 June 2007. https://martinfowler.com/bliki/DesignStaminaHypothesis.html. Accessed 1 November 2026; full web essay.

[32] M. Fowler. Some thoughts on LLMs and software development. martinfowler.com, 28 August 2025. https://martinfowler.com/articles/202508-ai-thoughts.html. Accessed 1 November 2026; brief excerpt.

[33] J. Gall. *General Systemantics: An Essay on How Systems Work, and Especially How They Fail.* General Systemantics Press, Ann Arbor, MI, 1975.

[34] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1995. ISBN 978-0-201-63361-0. Publisher's preface and Adapter excerpt: https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610; https://www.informit.com/articles/article.aspx?p=1398600. Accessed 1 November 2026; publisher description, not the full book.

[35] D. Gautam, S. Garg, J. Jang, N. Sundaresan, and R. Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2503.07832.

[36] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents? arXiv:2602.11988v3, 29 September 2026. https://arxiv.org/abs/2602.11988v3.

[37] C. A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*, volume I, pages 1–20. Reserve Bank of Australia, Sydney, 1975.

[38] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *ICLR*, 2024. Inspected text: arXiv:2305.11738v4, Section 3; accessed 1 November 2026. https://arxiv.org/abs/2305.11738v4.

[39] A. Herdağdelen, H. Joren, and A. Tamkin. How AI agents shape experts' judgment calls. Anthropic research article, 27 October 2026; accessed 29 October 2026. https://www.anthropic.com/research/agents-judgment-calls. Accessed 1 November 2026; study design, results, limitations and proposed safeguards.

[40] I. Herraiz, D. Rodriguez, G. Robles, and J. M. Gonzalez-Barahona. The evolution of the laws of software evolution: A discussion based on a systematic literature review. *ACM Computing Surveys*, 46(2):28, 2013. doi:10.1145/2543581.2543595.

[41] Ryan Hoque, Ashwin Balakrishna, Ellen Novoseller, Albert Wilcox, Daniel S. Brown, and Ken Goldberg. ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 598–608. PMLR, 2022. URL https://proceedings.mlr.press/v164/hoque22a.html.

[42] A. Hunt and D. Thomas. *The Pragmatic Programmer: From Journeyman to Master.* Addison-Wesley, 1999.

[43] Y. Jiang, K. C. Black, G. Geng, D. Park, J. Zou, A. Y. Ng, and J. H. Chen. MedAgentBench: A virtual EHR environment to benchmark medical LLM agents. *NEJM AI*, 2(9):AIdbp2500144, 2025. doi:10.1056/AIdbp2500144. Inspected text: arXiv:2501.14654v2; accessed 1 November 2026. Publisher full text unavailable.

[44] Z. Jiang, F. Huang, H. Xing, X. Wu, Y. Gao, R. Cao, M. Wang, S. Liu, and Y. Li. Demystifying agent skills: Why they work–until they don't. arXiv preprint arXiv:2608.14036v1, 2026; accessed 29 October 2026. https://arxiv.org/html/2608.14036v1.

[45] T. Kassis, V. Agarwal, Y. He, D. Patel, and A. M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents. arXiv:2609.00065, 2026. doi:10.48550/arXiv.2609.00065.

[46] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. HG-DAgger: Interactive imitation learning with human experts. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8077–8083, 2019. First posted 2018.

[47] M. Kim, T. Zimmermann, and N. Nagappan. An empirical study of refactoring challenges and benefits at Microsoft. *IEEE Transactions on Software Engineering*, 40(7):633–649, 2014. doi:10.1109/TSE.2014.2318734.

[48] A. Kumar, Y. Bajpai, S. Gulwani, G. Soares, and E. Murphy-Hill. Why AI agents still need you: Findings from developer-agent collaborations in the wild. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 432–444, 2025. arXiv:2506.12347v3, 9 October 2025.

[49] P. Laban, H. Hayashi, Y. Zhou, and J. Neville. LLMs get lost in multi-turn conversation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.06120v1, 9 May 2025.

[50] H.-P. Lee, A. Sarkar, L. Tankelevitch, I. Drosos, S. Rintel, R. Banks, and N. Wilson. The impact of generative AI on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–22. ACM, 2025. doi:10.1145/3706598.3713778.

[51] M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980. doi:10.1109/PROC.1980.11805.

[52] M. M. Lehman, J. F. Ramil, P. D. Wernick, D. E. Perry, and W. M. Turski. Metrics and laws of software evolution: The nineties view. In *Proceedings of the 4th International Software Metrics Symposium (METRICS '97)*. IEEE, 1997. doi:10.1109/METRIC.1997.637156.

[53] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. AIDev: Studying AI coding agents on GitHub. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2026. doi:10.1145/3793302.3797249.

[54] R. Lopopolo. Harness engineering: Leveraging Codex in an agent-first world. OpenAI, 11 February 2026. https://openai.com/index/harness-engineering/. Retrieval attempted 1 November 2026; HTTP 403, fresh full text unavailable.

[55] J. L. Lulla, M. Galster, J. M. Zhang, S. Baltes, and C. Treude. The impact of configuring agentic AI coding tools on build-vs-buy decisions: A study protocol. ESEM 2026, Registered Reports track. arXiv:2606.03907, 2026.

[56] Y. Ma, B. Kereopa-Yorke, and B. Schultz. Building to the test: Coding agents deliver what you check, not what you requested. arXiv preprint arXiv:2606.28430v1, 26 June 2026. https://arxiv.org/abs/2606.28430v1.

[57] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. arXiv:2303.17651v2, 2023; Section 2 inspected 1 November 2026. https://arxiv.org/abs/2303.17651v2.

[58] R. C. Martin. Why hardcore technical focus still matters big time. *Product Engineers* podcast, 29 September 2026. https://productengineers.com/episodes/season2/why-hardcore-technical-focus-still-matters-big-time-uncle-bob/. Accessed 1 November 2026; episode description, not the complete audio.

[59] S. Mhatre, Y. Bajpai, S. Gulwani, E. Murphy-Hill, and G. Soares. SWE-Sharp-Bench: A reproducible benchmark for C# software engineering tasks. In *International Conference on AI-powered Software (AIware)*, 2025. Accessed arXiv:2511.02352v2, 5 November 2025.

[60] Microsoft Research. Systematic debugging for AI agents: Introducing the AgentRx framework. First-party research blog, 12 March 2026; accessed 27 October 2026. https://www.microsoft.com/en-us/research/blog/systematic-debugging-for-ai-agents-introducing-the-agentrx-framework/. Accessed 1 November 2026; first-party summary excerpt.

[61] Microsoft. HVE Core: Hypervelocity Engineering Core. GitHub repository and documentation, tag `plugins-v3.3.106`, accessed 24 October 2026. https://github.com/microsoft/hve-core/tree/plugins-v3.3.106; https://microsoft.github.io/hve-core/. Tag recheck attempted 1 November 2026; organization SAML denied access, no bypass attempted.

[62] Microsoft Research. SkillOpt: Agent skills as trainable parameters. First-party research blog, 30 June 2026; accessed 27 October 2026. https://www.microsoft.com/en-us/research/blog/skillopt-agent-skills-as-trainable-parameters/. Accessed 1 November 2026; first-party summary excerpt.

[63] S. Mohsenimofidi, M. Galster, C. Treude, and S. Baltes. Context engineering for AI agents in open-source software. In *MSR '26*, pages 194–198, 2026. doi:10.1145/3793302.3793350. Inspected text: arXiv:2510.21413v4. https://arxiv.org/abs/2510.21413v4.

[64] Emerson Murphy-Hill, Jenna Butler, and Alexandra Savelieva. Adoption and impact of command-line AI coding agents: A study of Microsoft's early 2026 rollout of Claude Code and GitHub Copilot CLI, 2026.

[65] P. Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985. doi:10.1016/0165-6074(85)90032-8.

[66] A. Ng. Agentic design patterns part 2: Reflection. *The Batch*, DeepLearning.AI, 27 March 2024; accessed 1 November 2026. Practitioner letter, not a controlled study. https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-2-reflection/.

[67] M. Nigh. How to write a great agents.md: Lessons from over 2,500 repositories. The GitHub Blog, 19 November 2025, updated 25 November 2025. https://github.blog/ai-and-ml/github-copilot/how-to-write-a-great-agents-md-lessons-from-over-2500-repositories/. Accessed 1 November 2026; brief excerpt.

[68] G. Orlanski, D. Roy, A. Yun, C. Shin, A. Gu, A. Ge, D. Adila, N. Roberts, F. Sala, and A. Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755v2, 2026. Conference status requires separate verification; this citation pins the inspected preprint version.

[69] R. Pan, J. Zhang, L. Wu, Y. Zhang, H. Zhang, and L. Zhang. Do modifications made by coding agents make subsequent software maintenance harder? arXiv preprint arXiv:2610.14768v1, 10 October 2026.

[70] S. Park, W. Kim, R. Tan, J. Zhang, W.-S. Han, P. Gao, C. Park, Y. Yao, R. Fu, E. Nallipogu, Q. Lin, S. Rajmohan, and D. Zhang. AutoSaddler: Automatic harness optimization with durable updates from agent execution traces. arXiv:2608.23041v1, 24 August 2026; abstract inspected 1 November 2026. https://arxiv.org/abs/2608.23041v1.

[71] D. L. Parnas. Software aging. In *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, pages 279–287. IEEE, 1994. doi:10.1109/ICSE.1994.296790.

[72] A. Pavlenko, B. Crivat, B. Haynes, C. Curino, F. Psallidas, J. Slawinski, J. Freischuetz, L. Pereira Sanchez, M. Weimer, M. Demarne, M. Jasny, M. Gandhi, M. Bovykin, M. Milletari, P. Ghosh, Q. Bai, R. Ramakrishnan, R. Pandita, S. Matusevich, S. Venkataraman, S. Krishnan, M. T. Mahmood, T. Bang, V. Emani, X. Zhao, and Y. Zhu. Towards an AI software factory for data systems. arXiv:2609.36323v1, 28 September 2026. https://arxiv.org/abs/2609.36323v1.

[73] Sien Reeve O. Peralta, Fumika Hoshi, Hironori Washizaki, Naoyasu Ubayashi, Inase Kondo, Yoshiki Higo, Hiroki Mukai, Norihiro Yoshida, Kazuki Kusama, Hidetake Tanaka, and Youmei Fan. Why are agentic pull requests merged or rejected? an empirical study. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, pages 787–791, 2026. doi:10.1145/3793302.3793575.

[74] philia (uploader). Prompting Is Dead in 6 Months. Andrew Ng, Stanford. Third-party YouTube upload, 4 September 2026; automatic English captions inspected 1 November 2026. Ng introduces Laurence Moroney at 16:31, who gives the later technical-debt and reflection discussion. No caption support for the headline's six-month prediction. https://www.youtube.com/watch?v=9EuNUe-CJRM.

[75] PhotoCraft contributors. PhotoCraft: Source, parity records and public tracker observations. GitHub, source commit `5bb8536`; retrieved 25 October 2026. https://github.com/storytold/photocraft/tree/5bb853654196eca807212ff8955ba2eb293071ea. Dated counts and claim locators in the ancillary repository-observation record. Commit metadata accessed 1 November 2026 through GitHub CLI; application not executed.

[76] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield. Effective context engineering for AI agents. Anthropic, 29 September 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents. Accessed 1 November 2026; brief excerpt at redirected official page.

[77] P. Ralph et al. Empirical standards for software engineering research. arXiv:2010.03525v2, 2021 (first posted 2020). ACM SIGSOFT Empirical Standards: https://www2.sigsoft.org/EmpiricalStandards/. Accessed 1 November 2026; standards landing-page excerpt.

[78] W. W. Royce. Managing the development of large software systems. In *Proceedings of IEEE WESCON*, pages 1–9, August 1970.

[79] P. Runeson and M. Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi:10.1007/s10664-008-9102-8.

[80] J. M. Servera Bondroit. AMA Case Study 48: research and solution design for the Zava Forward retrofit programme. GitHub repository, commit 29c6b51, 24 October 2026. https://github.com/jmservera/ama. Repository/commit and named issue/PR metadata accessed 1 November 2026; historical live journey unverified.

[81] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023. Inspected text: arXiv:2303.11366v4, Section 3; accessed 1 November 2026.

[82] A. Simkute, L. Tankelevitch, V. Kewenig, A. E. Scott, A. Sellen, and S. Rintel. Ironies of generative AI: Understanding and mitigating productivity loss in human-AI interaction. *International Journal of Human–Computer Interaction*, 41(5):2898–2919, 2025. doi:10.1080/10447318.2024.2405782.

[83] A. Sinha, A. Arun, S. Goel, S. Staab, and J. Geiping. The illusion of diminishing returns: Measuring long horizon execution in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.09677v3, 13 March 2026.

[84] M. Strathern. 'Improving ratings': Audit in the British university system. *European Review*, 5(3):305–321, 1997.

[85] L. Tankelevitch, V. Kewenig, A. Simkute, A. E. Scott, A. Sarkar, A. Sellen, and S. Rintel. The metacognitive demands and opportunities of generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pp. 1–24. ACM, 2024. doi:10.1145/3613904.3642902.

[86] A. Tornhill and M. Borg. Code red: The business impact of code quality. A quantitative study of 39 proprietary production codebases. In *Proceedings of the International Conference on Technical Debt (TechDebt '22)*, pages 11–20. ACM, 2022. doi:10.1145/3524843.3528091.

[87] U.S. Securities and Exchange Commission. In the matter of Knight Capital Americas LLC. Order, Exchange Act Release 34-70694, 16 October 2013. https://www.sec.gov/files/litigation/admin/2013/34-70694.pdf. Order retrieval attempted 1 November 2026; HTTP 403, fresh full text unavailable.

[88] M. Waseem, A. Ahmad, K.-K. Kemell, J. Rasku, S. Lahti, K. Mäkelä, and P. Abrahamsson. Vibe coding in practice: Flow, technical debt, and guidelines for sustainable use. arXiv:2512.11922, 2025.

[89] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi:10.1007/978-3-642-29044-2.

[90] C. Yang, H. He, T.-C. Cheng, J. Butler, E. Murphy-Hill, C. Badea, and C. Bird. (When) should a software factory be lights-off? Benchmarking code review agents on bug detection. arXiv preprint arXiv:2610.13668v1, 9 October 2026. https://arxiv.org/abs/2610.13668v1.

[91] J. Young. Effective harnesses for long-running agents. Anthropic, 26 November 2025. https://www.anthropic.com/research/effective-harnesses-for-long-running-agents. Accessed 1 November 2026; brief excerpt, not a fresh complete study audit.

[92] H. Yu, L. Liu, X. Jiang, Y. Jia, S. Wang, P. Qian, and Y. Chen. Habituation at the gate: Rising approval and declining scrutiny in human review of AI agent code. KDD 2026 Workshop on Agentic Software Engineering, 2026. arXiv:2606.22721.

[93] M. Zhang, O. Press, W. Merrill, A. Liu, and N. A. Smith. How language model hallucinations can snowball. In *ICML*, PMLR 235:59670–59684, 2024. Inspected original text: arXiv:2305.13534v1; https://proceedings.mlr.press/v235/zhang24ay.html.

[94] Z. Zhong, A. Raghunathan, and N. Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.20270.

A Supplement

This section supplies source-specific qualifications, reproducibility detail and unperformed discriminating checks beyond the main account. It does not repeat the Background, case chronology or figures. The evolving protocol and current registered-base/unregistered-amendment status are in Section 7; the details below retain their individual source and measurement boundaries, including historical freeze provenance.

A.1 Additional source qualifications

The details below qualify specific uses of the literature in Section 2; they are not additional results from this study.

Table 12: Source-specific qualifications relevant to the proposed measurements.

Source	Additional detail	Consequence for interpretation
Sinha et al. [83, Sec. 3.2, Figs. 5–6]	The artificial-history experiment varies erroneous prior updates. Qwen3 with thinking enabled avoids the reported effect in that setup, whereas scaling non-thinking models does not remove it.	Earlier-error sensitivity depends on configuration; it is not a universal property of all models or a measured repository-decay rate.
Pan et al. [69, Sec. 5.1, Fig. 4]	In the `parse_crontab` episode, a local Python 3 change satisfies current tests but omits module-wide compatibility aliases later code uses.	An initially successful patch can leave a latent dependency failure. This complements, rather than repeats, the line-terminator example in the main text.
Gloaguen et al. [36, Secs. 4.2–4.4]	File length does not explain success in their analysis, and generated context helps when other documentation is removed.	Instruction length alone is not an adequate exposure or burden measure; available alternative documentation matters.
Jiang et al. [44, Secs. 5.2–5.5]	Skill misapplication and missing runtime verification remain failure modes. The retrieval diagnostic is separate from downstream execution.	Lower exact-match access alongside sustained task success does not isolate an effect of retrieval; accessed, applied and sufficient guidance differ.

The provenance of merge-outcome studies also matters. Peralta et al. start with 9,799 human-reviewed agent PRs and manually inspect 717. Their 35.7% clear-failure share among rejections and 15.4% explicit reviewer-involvement share among merges describe that manual sample [73]. Murphy-Hill et al.'s roughly 24% higher merge count among adopters is an observational rollout estimate, not delivered value [64]. AIDev's 932,791 agent-authored PRs from five agents across 116,211 repositories provide a population for research, not a count of accepted user outcomes [53].

Pavlenko et al.'s World Model offers an additional context-management design: preserve raw evidence, derive rebuildable views and retrieve compact, task-specific context with provenance and access controls [72]. That architecture is an alternative to relying on an expanding read path; its effect on the AMA case is untested.

A.2 Case rival-explanation register

The main interpretation is in Section 5.2. The register preserves recorded rivals separately from untested alternatives. A limit is not grounds for ruling an explanation out. Score/deployment correspondence remains unverified (Section 3.6).

Table 13: Rival explanations with direct case records.

Explanation	Case support	Counter-evidence or limits	Treatment
Migration decision	ADR 0006 (issue #196) froze feature work in the MVP and prioritised a new layout.	Health and restart checks passed, not usability.	Migration and coordination changed together; autonomy alone is not identified.
Capstone rubric	Research and documentation are rewarded; full completeness marks coexist with blocked sign-in.	A working demonstrator is required; release/scope linkage is pending.	Possible incentive, not a measured effect.
Owner overrides and priorities	Committed RPI records, full gates and delegated merging; PR #445 required an exception.	The MVP used agents successfully; HVE Core defaults retain human stops.	Owner choices are part of the case.
Model and tool changes	Versions changed during the study.	No matched rerun under fixed versions.	Actual session configuration remains partly unknown.
CI and cloud budgets	Issue #413 moved validation locally; costs prompted a higher ceiling and reduced resources.	Merges continued without established deployed usability.	More budget is not proven to resolve delivery.
Identity and sign-in setup	Missing registration/staff bindings; issue #349 distinguishes redirect from sign-in.	Those steps worked in the MVP.	Concrete configuration obstacle, not proof of code decay or lost knowledge.

Table 14: Additional plausible rivals and unperformed discriminating checks.

Explanation	Reason considered	Counter-evidence or limits	Evidence still needed
External services and deployment drift	Model/identity configuration differs from the MVP.	No provider outage is established; health checks were narrower.	Versioned deployment/configuration and dependency-health records for the journey.
Scope expansion	RAI and consumption criteria added to issues; further resident-preview obligations.	Expansion predates autonomy; some controls are justified.	Original/added scope, sequencing and time-to-outcome comparison.
Inadequate acceptance tests	Checks passed without completed real-user sign-in; scorer read artefacts.	Bounded, honest tests need not be poor tests.	Capability-to-task-oracle mapping and execution evidence.
Rate and context limits	Parallel sessions and a large read path.	No runtime rate-limit, truncation or attention dataset analysed.	Actual inputs and blocked/retried request events.
Specification ambiguity	Selective review of long, cross-referenced requirements.	Important domain/consent decisions survived; ambiguity not independently coded.	Independent interpretation and conflicting acceptance examples.
Ordinary prototype debt	Rapid MVP followed by restructuring and rebuilding.	MVP worked intermittently; no comparable human-led migration.	Failure analysis distinguishing inherited debt from introduced faults.

A.3 Candidate-mechanism evidence audits

These are operational boundaries for this paper, not validated necessary and sufficient conditions for all agentic development. The indicators are already reported in Table 4, Table 1 and the case chronology; they are not repeated here. Experimental implications state what evidence would

discriminate an explanation; the five-arm experiment does not manipulate the five candidates separately.

A.3.1 Specification inflation

Element	Audit
Definition	Requirements and acceptance obligations expand faster than independently validated behaviour, without evidence that the added scope is needed for the next usable increment. Length alone is insufficient.
Counter-evidence	Growth began before autonomy and sometimes followed the owner's feedback; regulatory and consent controls can justify detail.
Rival explanations	Genuine scope expansion, specification ambiguity, capstone incentives and an ambitious migration rather than agent-driven inflation.
Experimental implication	Compare smaller requirements horizons at matched scope and resources. The current fixed-suite experiment cannot identify this mechanism, even if consolidation helps.

A.3.2 Proxy capture

Element	Audit
Definition	A completion proxy becomes a decision criterion and remains positive when an independent test of the intended outcome is negative. Proxy use alone does not establish capture.
Counter-evidence	Agents explicitly reported unmet acceptance; neither fabricated approval nor deliberate deception is established.
Rival explanations	An inadequate scorer or acceptance oracle, different evaluation targets, and a deployment failure invisible to repository review.
Experimental implication	Disagreement between visible completion and hidden behavioural tests would expose proxy limits. It would not by itself establish intentional test gaming or explain why a gate was chosen.

A.3.3 Learning evaporation

Element	Audit
Definition	A recorded lesson relevant to a later decision is absent from, unusable on, or not applied through that decision's effective context path, and the associated problem recurs. A missing citation alone is insufficient.
Counter-evidence	Important domain decisions survived; the lesson could have been known but deferred. Actual session attention was not measured.
Rival explanations	Deliberate migration sequencing, deployment-specific configuration, context limits, or conflicting requirements rather than lost knowledge.
Experimental implication	Trace actual inputs and repeated faults. B–C cannot isolate memory quality because DESIGN and REVIEW notes and acting permissions differ. A matched-context study would be needed.

A.3.4 Process-gate inflation

Element	Audit
Definition	Additional procedural obligations increase per-change burden without demonstrated proportional improvement in the targeted outcome. More gates alone are not evidence that their net benefit is negative.
Counter-evidence	Checks caught bounded defects and enforced evidence integrity. Gate duration and human review effort were not measured per PR.
Rival explanations	Legitimate safety and privacy requirements, budget constraints, new deployment complexity and owner-selected risk tolerance.
Experimental implication	Measure gate time and accepted outcomes under matched risk. The pilot's resource description concerns the package, not the effect of removing this repository's required gates.

A.3.5 Feedback starvation

Element	Audit
Definition	Independent outcome-level feedback is insufficient to correct delivery decisions despite abundant artefact-level validation. The defining gap is between the evaluated proxy and intended outcome, not test count alone.
Counter-evidence	Human oversight continued; agents surfaced blockers; engineering checks provided real, though narrower, feedback.
Rival explanations	Identity and service configuration, migration priority, incomplete acceptance coverage, prototype debt or insufficient resources.
Experimental implication	Hidden retention tests and CR1 supply behavioural feedback to the evaluator. The experiment does not test live deployment recovery, product selection or a human-use checkpoint cadence.

A.4 Measures and lexicon

Log entries. The knowledge-base log (`docs/reference/log.md`) was split into entries at each second-level heading that begins with a date. An entry belongs to the date in its heading. Word counts use whitespace tokenisation of the entry, heading included.

Evidence-boundary phrases. Each entry was searched, case-insensitively and on word boundaries, for these patterns: *unmet*; *not accepted*; *not inferred*; *is inferred*; *remain(s) open*; *remain(s) unproved*; *remain(s) unverified*; *not claimed*; *not established*; and *no* followed, within forty letters, spaces, commas, slashes or hyphens, by *is* or *was* and then *claimed* or *inferred*. Density is matches per 1,000 words.

Reference tracing. An entry cites the simplicity requirements if it contains an identifier from SA-001 to SA-010 or the word *simplicity*. It cites the development profile if it contains *deploymentProfile*, *development profile* or *allowPublicNetworkAccess*.

Repository counts. Issue and pull-request counts come from the GitHub search API on 24 October 2026 (for example, `is:pr is:merged merged:>=2026-10-15`). Document sizes are whitespace word counts of the files at commit `29c6b51`.

Proposed measures. *Time to first human use*: elapsed time from the start of a delivery cycle to the first time a person uses the running increment for its purpose. *Age of the last usable increment*: time since a person last verified that the deployed system could be used for its main journey. *Behaviour-changing merge ratio*: the share of merged pull requests that change what

a user can observe in the running increment. *Cost per accepted outcome*: total model, tool and cloud cost divided by the number of outcomes a human accepted in the same period. Predeclare the outcome unit, counting rule, attribution and period. With zero accepted outcomes the ratio is undefined; report the total cost and zero count.

A.5 Consolidation step prompt

Verbatim frozen v4 prompt from execution source e8707ecc8965d3f81f5142a2603042bfca1aab15, not the amended study's approved prompt. The archived source is byte-identical, SHA-256 75f646edbe04e1e3403b092b13edd6b66615a3362e67ad49744d229cdcb94f29. Display wrapping does not alter its text. Braces mark runner parameters.

```
<!-- markdownlint-disable-file -->
# Consolidation step prompt (arm B)

Do not add or change features. Your job is to leave the code better than you found it without
    changing its behaviour.

1. Run `make check`, then read reports/quality.md (functions above complexity 10, duplicated
    blocks, lint warnings, file sizes).
2. Review the last feature's diff (`git diff feature-start feature-end`) and the hotspots
    against this checklist:
   * duplicated logic;
   * long or complex functions;
   * unclear names;
   * dead code;
   * responsibilities in the wrong place, or rules from one feature leaking into another;
   * tests that check implementation details instead of behaviour.
3. Make small, behaviour-preserving changes. Apply every change through `./tcr "<message>"`: it
    runs all tests and commits if they pass, or reverts if they fail.
4. Never modify or delete files in tests/examples/. You may add tests that pin current behaviour
     before you refactor.
5. Rewrite DESIGN.md in at most 300 words: the current structure, where each business rule lives
    , and known debts. Replace its content; do not append.
6. Stop when the quality budget is met (no function above complexity 10, duplicated lines below
    3%, zero warnings in the configured lint rules) or when your time budget ({
    consolidation_budget}) is used.
7. Finish with a summary of at most five lines: what you changed and what debt remains.
```

A.6 Deferred intervention measures

The study excludes intervention-rate outcomes but retains help and administrative records for eligibility and cost. These definitions are proposed future measures, not collected results. HG-DAgger and ThriftyDAgger supply cross-domain measurement precedents, not coding-agent effectiveness evidence [41, 46].

For run r, $n_r \geq 1$ is attempted feature cycles, including assigned post-steps and failed cycles, and h_{rj} qualifying corrective-help events. The full-run rates below require complete help annotation for every attempted cycle. Otherwise report annotation coverage and leave the full-run rate unknown. Observed human intervention rate would be

$$\mathrm{HIR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} \mathbf{1}\{h_{rj} > 0\}. \tag{5}$$

Event counts and active human minutes are separate; repeated help in one cycle counts once for incidence. Let $d_{rj} \in \{0, 1\}$ indicate an explicit help request or a block needing corrective human action. Then

$$\mathrm{IDR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} d_{rj}. \tag{6}$$

Requests and supplied help differ. A timeout alone is not a help request; zero attempts gives undefined rates. A future annotation protocol needs event/cycle/phase locators, coverage checks and adjudication; absent records cannot be zero. Routine launches, planned approvals and safety stops differ from corrective rescue. Low intervention must not reward silent failure or discourage necessary human judgement.

A.7 Historical freeze provenance

The reported case-tool snapshot is HVE Core `plugins-v3.3.106`, snapshot commit 2df5013e29 ebfd2ce61bcd0cf021d8c8186482eb, generated from source e9af018fe5ffbfbc72321a8bca21fb987 359af08. These are reported provenance identifiers, not verification of installation date or every session's runtime.

The historical v4 execution source was e8707ecc8965d3f81f5142a2603042bfca1aab15, with manifest-only commit 03c50856838b539780d57216af2a4cf6744187d7. Its attached osf-v4-202 6-10-31.tar.gz checksum was 1a42fac40a4f1aa82df59c8c3f7f8d6dce37d22502c5952c398362ac63 d6bd65. The runtime image was sha256:090220b0b172ec205f1ea473d7c3a223eec5d7d619eb21 acdcf15c9289287fa3. These identify historical inputs, not a freeze or approval of the five-arm amendment. The original registration at https://osf.io/ywsrv/ is owner-reported approved after submission on 28 October; the inspected v4 prospective amendment flag was false and its completion date null. New approval and actual prospective recording cannot be inferred or backdated. The later registered base at https://osf.io/xcpt4 and current disclosed unregistered execution are described in Section 7.1 and Data availability. They do not rewrite these historical freeze records.

A.8 Measurement fields and provenance

The following inventory is a read-only account of public source e71037d961cb80fc5c65943779 6de7736820f6c9 in https://github.com/jmservera/ama-thought-mve, not a remeasurement of private experimental data. The collection path is harness/runner.py → harness/measure.py → skeleton/tools/metrics_core.py; pilot outcomes are derived in harness/pilot_analysis.py, with consumption parsing in harness/analyze.py. At this running-source pin, the numerical evaluator does not invoke an AI judge. The separate post-run instrument below adds supplementary ratings without changing that collection path.

Tool settings and repeatability. The public host configuration requests Lizard, jscpd and Ruff; per-run configuration and version records, not package presence alone, establish the tools actually used. The declared pins are Lizard 1.24.3, Ruff 0.17.3 and jscpd 5.4.1. The code records backend names and supports explicitly different internal approximations; these are not interchangeable instruments. Lizard measures each Python source function. jscpd runs from an empty configuration directory in Python mode with minimum 50 tokens and five lines. Ruff runs with isolated configuration, fixed `E,F,W,C90,B,SIM` selection, JSON output, no cache and `-exit-zero`; diagnostics are counted rather than treating its exit code as a clean-lint verdict.

The actual invocation omits `-ignore-noqa`, so inline suppressions remain effective. No suppression-count field or maintainability-index field appears in the active dictionary. An installed dependency or qualitative review phrase must not be reported as a measured variable. With no detected functions the engine emits zero mean/maximum/share summaries; these are empty-input sentinels, not measured zero complexity, and the mean/share denominator is undefined. The measurement engine, collector, pilot analyzer, failure-attribution module and C prompt have identical source blobs in the three disclosed implementation epochs. This does not make the whole evolving dataset a single frozen implementation or establish equal execution conditions.

Table 15: Active raw fields, scales and scope. Missing measurements are not zero. Structural counts and model-generated notes are not independent efficacy outcomes.

Field or record	Definition and reporting limit
`hidden`, `hidden_passed`, `hidden_total`	Fixed test identities with passed/failed/error/skipped/timeout outcomes, pass count and discovered denominator. Final coverage requires the exact feature panel and complete binary endpoint results.
`open_losses`, `density`, `episodes`	Counts and fraction failing among previously acquired unchanged contracts; onset, recovery, recurrence and uncertainty. Conservative current summaries can be unavailable when acquisition or observations are incomplete.
`cr_success`, `cr1_cause`	Composite success over CR1 and retained passing feature contracts, plus recorded cause/disposition. An explicitly evidenced terminal agent failure can resolve an otherwise unavailable success as false; this is not an AI-generated quality grade.
`mean_ccn`, `max_ccn`, `share_ccn_over_10`	Unweighted mean and maximum per-function cyclomatic complexity; fraction of source functions above 10, not percentage unless multiplied by 100. Mean and fraction are rounded to three and four decimals respectively.
`dup_pct`	The clone detector's duplicated-line percentage. jscpd's line denominator is not automatically the independently counted source NLOC denominator.
`lint_warnings`, `lint_per_kloc`	Diagnostic count and $1000 \times$ warnings/source NLOC; the latter is undefined for an empty source denominator. No separate suppression count is emitted.
`nloc_src`, `nloc_tests`, `files_src`, `functions`	Python files under source/test directories; lines excluding blank and full-line comments, source-file and function counts. These are not instrumented test-coverage measures.
`md_words`, `test_counts`, `diff`, `commits`	Whitespace-delimited Markdown words excluding generated reports/dot directories; AST test/assertion counts; Git additions/deletions by area and post-step commits. Counts describe artifacts, not semantic quality.
Usage and timing	`totalNanoAiu`/10^9 supplies AI credits; input, output, cache-read and cache-write token counts remain separate. `agent.seconds` is host-observed session wall time; `totalApiDurationMs` is available API duration, not necessarily provider-active time. Unknown or incomplete usage cannot become zero-cost work.
Integrity and assistance	Example-file hashes, protected-test changes, violations and heuristic leak flags are recorded. Explicit annotation/evidence determines assisted, unassisted or unknown status; no help is inferred from silence. HIR/IDR remain excluded/deferred, not automatic zero-valued scores.

From observations to summaries. The collector evaluates at feature and retained post-step checkpoints, and again after CR1. A/D have no separate post-step; B/C/E retain their assigned post-step. It writes outcomes, provenance, source HEAD, raw metrics, integrity and diff records. The current final score is unavailable when sequence/panel identities or binary endpoint observations are incomplete. The loss/CR1 reporting gaps in Section 7.4 remain explicit rather than being filled with invented data. Credits require complete consumption flags; all-attempt delivery/CR1 totals need a matching attempt journal. Aggregate statistics and seeded resampling operate on these records, not on AI opinions. Model-specific allocation, missingness and observed runtime variation remain distinct from whether the scoring algorithm is programmatic.

A.9 Supplementary judging instrument and interpretation

The proposed post-collection instrument is documented in judging/README.md at b7973e20 0a6edbb1d19355badae3b7e03a066b09 in the experiment repository. The README's Git blob is 6cc0c236e4ca093198d38909dceb80f850196faa. This pin identifies a separate development branch, not a replacement for the running experiment or evidence that judging has occurred. Preparation will use a stopped local archive, retain every allocated model/arm/replicate cell and select recorded measurement HEADs. Missing P_{10} or CR1 observations cannot be replaced by an earlier successful checkpoint. Committed runtime assets will remain available in a read-only isolated evaluator with an inspected immutable image. Each scenario starts with a fresh in-memory Workbench.

Table 16: Fixed supplementary workflow panel. Feature labels denote dependencies, not attribution of a failure to a particular feature.

Workflow	Exercised requirements
F1	Registration, combined filtering, atomic import and recovery
F2	Lifecycle, invalid transitions, history and stage counts
F3	Handoff, decline and acceptance with ownership preservation
F4	Rework and reopening with ownership and reasons retained
F5	Safety holds, permitted rework and role-controlled release
F6	Consent withdrawal, reinstatement and safety precedence
F7	Offered/confirmed appointments, verified escalation and reopening reset
F8	Live contractor reads, protected fields and read-only enforcement
F9	Stable priority ranking with interacting hold, consent and vulnerability reasons
F10	Audit verification, tamper detection and visibility-preserving export
Journey	F1–F10 delivery/reopening with consent, handoff, appointments and audit
CR1	Region-scoped reads, rankings, exports and approved cross-region handoff

Witnessed endpoint loss. For trajectory r and endpoint $e \in \{P_{10}, \mathrm{CR1}\}$, let $\mathcal{W}_{r,e}$ contain panel scenarios witnessed passing at an earlier eligible end-of-cycle checkpoint, and let $S_{r,e}(w)$ be the observed binary endpoint outcome for scenario w. When $|\mathcal{W}_{r,e}| > 0$ and every member has an available endpoint, the supplementary retention density is

$$d_{r,e}^{\text{workflow}} = \frac{\sum_{w \in \mathcal{W}_{r,e}} \mathbf{1}\{S_{r,e}(w) = 0\}}{|\mathcal{W}_{r,e}|}. \tag{7}$$

It is a dimensionless fraction of witnessed earlier capabilities lost at the selected endpoint, ranging from zero to one. An empty risk set or any unavailable risk-set endpoint leaves the density undefined. Publish the witnessed risk-set size, losses and unavailable count with final scenario outcomes. Acquisition itself depends on treatment: low loss among a small acquired set can accompany poor final fulfilment. This conditional density therefore cannot alone rank treatment efficacy or replace the original hidden-contract loss measure.

The instrument distinguishes retained and observed-loss scenarios, endpoint passes without an earlier witness, complete histories with no observed pass, incomplete histories with no observed pass, and unavailable endpoints. F10 first becomes eligible at P_{10}, so a failure there cannot establish earlier-capability loss; its P_{10} pass can establish a post-CR1 baseline. The full delivery journey likewise first becomes eligible at P_{10}; CR1's region workflow first becomes eligible after CR1. The maximum earlier-witness risk sets therefore contain nine scenarios at P_{10} and eleven after CR1. Post-CR1 loss uses any earlier eligible P witness, so it can include a failure already present at P_{10}. An incremental P_{10}-to-CR1 loss would require a passing P_{10}

baseline and remains distinct from this endpoint measure. A witnessed pass followed by an endpoint failure supports endpoint loss despite missing intervening observations, while leaving onset time unresolved. A final pass can conceal a recovered regression. The panel estimates neither uninterrupted retention nor episode duration, recurrence or causal feature attribution.

Ratings, blinding and calibration. Functional completeness, workflow coherence, error recovery and API usability are separate ordinal dimensions: 0 absent/unusable; 1 major blocking failures; 2 material gaps; 3 working with minor evidenced limitations; 4 coherent fulfilment of exercised requirements. Unsupported dimensions are null, not zero, and every rating requires a rationale with citations to supplied final execution observations. There is no composite software-quality score. A perfect completeness rating is rejected when execution demonstrates a workflow failure.

Random packet identifiers and shuffled submission order conceal development arm/model labels. Judges receive requirements, rubric, response shape and execution evidence, excluding original run IDs, commit hashes, source trees, treatment notes, transcripts and usage logs. The inspected packet also includes an earlier passing witness and its retention label. Historical and final observations reuse scenario-local IDs; validation checks citation membership in final evidence, without establishing which occurrence influenced a rationale. Historical anchoring and dimension-specific support therefore need human inspection. Endpoint-only rating packets or checkpoint-qualified IDs would require a separately versioned instrument; they are not claimed implemented here. The separate judge adapter disables tool/configuration discovery and checks its tool-free transport. Residual performance patterns and returned values can still suggest treatment or model; the preparing researcher is not automatically blinded. Scenario instrumentation is a software-behaviour assessment, not proof against deliberate candidate tampering.

Before interpreting ratings, a blinded human-reviewed calibration sample must cover successful, incomplete and regressed workflows, with sample selection, reviewers, timing and disagreements recorded. Multiple judge models can expose model-specific disagreement; overlap with development models and possible family preference require disclosure. Fresh repeated sessions provide descriptive means and ranges, not established inter-rater reliability. Three repetitions in the example configuration are a planning example, not a validated reliability threshold. The retained ledger should also be examined by judge for abstentions and disagreement: pooled ranges combine within-judge repeat variation with between-judge differences.

Aggregation and resources. A scenario/dimension mean requires all scheduled judge/repeat ratings. A trajectory mean describes its available scenario means and retains available/scheduled counts and a completeness flag. Ordinal averaging is a pragmatic descriptive convention, without equal-interval calibration. Arm differences require complete trajectory scores for both compared arms in every allocated development model; otherwise the contrast is unavailable rather than estimated from complete survivors. Complete-frame contrasts equally weight within-model differences, as in Equation 2, with B–D the matched-ceiling package comparison and B–A, B–C and E–B secondary. The tool supplies no bootstrap intervals, hypothesis tests or formal verdicts. Repeated scenarios, snapshots and judge sessions remain nested within trajectory; implementation epochs and shared scheduling/service effects retain the unresolved interpretation limits in Section 7.6. The inspected report does not export block/epoch identifiers, so a private trajectory-to-source/epoch/block linkage is needed for those checks.

Before inspecting candidate outcomes, the researcher will record judging source/scenario/rubric hashes, judge runtime/model and adapter versions, repetitions, budget, missingness policy, calibration procedure and any prior exposure to experiment outcomes. Local timestamps and hashes support provenance, not independently verified preregistration. Preparation spends no AI credits. For N complete trajectories with CR1, J judge configurations and R repetitions, at most $2NJR$ sessions are scheduled for the two endpoints; wholly unavailable specimens

trigger no calls. Paid judging needs separate explicit operator authorisation. Recorded judge credits and unknown-charge reservations remain separate from development/treatment costs. Admission reservations do not establish a hard provider spending cap. Configuration and request hashes bind resume; interrupted, invalid, failed or overspent calls block new admission pending documented operator recovery rather than automatic replacement. Per-call admission also does not establish full-frame affordability; budget exhaustion can leave required comparisons unavailable. Calibration costs require their own allowance and accounting. Public reports require review and sanitisation; raw packets and mappings remain private.

A.10 Active AI review and consolidation prompt examples

These are the exact checked-in B/C templates at the active source commit, not a new judge prompt, a protocol amendment or a complete per-session provider request. The runner removes HTML-comment lines and the first Markdown heading, substitutes the configured post-step budget in minutes, and saves the actual supplied text as the step's `prompt.md`. The public configuration has a 900-second post-step budget, rendering "15 minutes"; this inspection does not verify every private run's rendered prompt or provider/system context. Each step is a fresh session under its assigned study model/effort. The model can reason qualitatively over the visible report and diff, but hidden outcomes are measured outside it.

Arm B requests behaviour-preserving changes and structural stopping targets. Arm C requests a 300-word issue-oriented `REVIEW.md` without code/test changes. The host enforces the permitted-file and retained-test rules; it does not parse either note into a numerical quality score. Reviewer disagreement is therefore treatment variation, not a measured inter-rater reliability or calibrated-judge result. The template's "optional placebo" heading is retained verbatim; this manuscript treats C as information-bearing review, not an inert placebo. The historical v4 prompt in Section A.5 remains a separate unchanged source record.

Arm B: consolidation. Source harness/prompts/consolidation-step.prompt.md; SHA-256 1ee3da712fcb787a7843d1270aabf61b55abb7c8b7df2534d4adc078b0241494. Display wrapping does not change the retained template bytes.

```
<!-- markdownlint-disable-file -->
# Consolidation step prompt (arm B)

Do not add or change features. Your job is to leave the code better than you found it without
    changing its behaviour.

1. Run `make check`, then read reports/quality.md (functions above complexity 10, duplicated
    blocks, lint warnings, file sizes).
2. Review the last feature's diff (`git diff feature-start feature-end`) and the hotspots
    against this checklist:
   * duplicated logic;
   * long or complex functions;
   * unclear names;
   * dead code;
   * responsibilities in the wrong place, or rules from one feature leaking into another;
   * tests that check implementation details instead of behaviour.
3. Make small, behaviour-preserving changes. Apply every change through `./tcr "<message>"`: it
    runs all tests and commits if they pass, or reverts if they fail.
4. Never modify or delete files in tests/examples/ or any accepted Python test file (including
    imports, fixtures and constants). Add tests that pin current behaviour only in new files.
5. Rewrite DESIGN.md in at most 300 words: the current structure, where each business rule lives
    , and known debts. Replace its content; do not append.
6. Stop when the quality budget is met (no function above complexity 10, duplicated lines below
    3%, zero warnings in the configured lint rules) or when your time budget ({
    consolidation_budget}) is used.
7. Finish with a summary of at most five lines: what you changed and what debt remains.
```

Arm C: read-only review. Source harness/prompts/review-only-step.prompt.md; SHA-256 06a8dfcaee30ab52c243c771844619cf08f23854ef866f1bfde5277362391725. This is a qualitative treatment instruction, not an outcome-grading rubric.

```
<!-- markdownlint-disable-file -->
# Review-only step prompt (optional placebo arm C)

Do not change any code or tests. Your job is to review what was just built.

1. Run `make check`, then read reports/quality.md.
2. Review the last feature's diff (`git diff feature-start feature-end`) and the hotspots
    against this checklist: duplicated logic; long or complex functions; unclear names; dead
    code; responsibilities in the wrong place; tests that check implementation details instead
    of behaviour.
3. Rewrite REVIEW.md in at most 300 words: the main issues you see and where they are. Replace
    its content; do not append. Do not modify any other file.
4. Time budget: {consolidation_budget}. Commit REVIEW.md and stop.
```